SITE-SAFE: A DIGITAL ECOSYSTEM FOR AUTONOMOUS CONSTRUCTION SAFETY

Muhammad Hamza

Hamza Hussain

Hanan Saeed

Supervised By

Muhammad Yousif

Submitted for the partial fulfillment of the BS Computer Science degree to the Faculty of Engineering & Computing

DEPARTMENT OF COMPUTER SCIENCE

NATIONAL UNIVERSITY OF MODERN LANGUAGES, LAHORE

April, 2026

ABSTRACT

Construction sites need evidence that safety controls were followed, yet manual inspections and passive CCTV provide only intermittent oversight. Site-Safe is a camera-first safety compliance system that detects hazards from site cameras, identifies the workers involved, alerts supervisors on web and mobile clients in real time and keeps an auditable per-worker record. This report describes its design, implementation and evaluation, and separates what was measured from what remains to be verified.

The implemented system combines a ten-class YOLO11-medium PPE detector that raises one alert per person per violation, QR badge identification that also places workers in a camera's zone, pose-based fall detection, per-person vehicle proximity warnings, danger areas drawn on the live camera picture, inactivity tracking and a trained fire and smoke detector whose severity follows hot-work permits. Camera events, with their evidence frame and model version, are buffered in a local outbox and posted with per-camera device tokens to a Django REST and WebSocket backend on PostgreSQL, which provides role-based access control, configurable alert rules, push and e-mail notification, zone head counts, automatic attendance and idle timelines, and DOCX and PDF reports that carry the evidence. A React command centre with a live camera grid and a site map showing every camera, and a React Native (Expo) mobile application with the same map and live pictures, are the clients. The Smart Vest's software path is implemented and exercised with a labelled simulator; the physical vest was moved to future work.

All 470 automated tests pass. On a single Apple M4 host, events reached the dashboard with a 95th-percentile latency of 42 ms, all 100 events queued during a five-minute outage were stored exactly once about 30 s after the API returned, a two-hour soak missed none of 1,431 probes, and the full detector set, including fire, ran at 16 frames per second on the laptop GPU. The PPE model measured mAP@0.5 of 0.562 on the held-out split, but missing-equipment classes remain weak (no-gloves 0.062, no-boots 0.130) and the published split places near-duplicate images in both training and test data, so the score is an upper bound; the fire and smoke model reached 0.787. The camera features were verified end to end with test videos; QR range tests, push delivery to a phone and footage from a real site are outstanding.

CERTIFICATE

Dated: _____

Final Approval

It is certified that the project report titled “Site-Safe: A Digital Ecosystem for Autonomous Construction Safety”, submitted by Muhammad Hamza, Hamza Hussain and Hanan Saeed for the partial fulfillment of the requirement of the Bachelor’s Degree in Computer Science, is approved.

COMMITTEE

Dean (Engineering & Computing): _____ Signature: _____

Head of Department, Computer Science: _____ Signature: _____

Project Coordinator: _____ Signature: _____

Supervisor: Muhammad Yousif Signature: _____

DECLARATION

We hereby declare that our dissertation is entirely our own work and genuine and original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group members:

Name	Signature
Muhammad Hamza	_____
Hamza Hussain	_____
Hanan Saeed	_____

PLAGIARISM CERTIFICATE

This is to certify that the project entitled “Site-Safe: A Digital Ecosystem for Autonomous Construction Safety”, which is being submitted herewith for the award of the Degree of Bachelor’s in Computer Science, is the result of the original work by Muhammad Hamza, Hamza Hussain and Hanan Saeed under my supervision and guidance. The work embodied in this project has not been done earlier as the basis for the award of any degree or comparable certificate or similar title by any other diploma or examining body or university, to the best of my knowledge and belief.

Turnitin similarity index: _____ %

Internet sources: _____ %

Publications: _____ %

Student papers: _____ %

Date: _____ Muhammad Yousif (Supervisor)

TURNITIN ORIGINALITY REPORT

Site-Safe: A Digital Ecosystem for Autonomous Construction Safety [BSCS] by Muhammad Hamza, Hamza Hussain and Hanan Saeed, from Muhammad Yousif.

Processed on: _____ ID: _____

Word count: _____ Similarity index: _____ %

[Insert the Turnitin originality report pages here.]

TABLE OF CONTENTS

Chapter 1: Introduction 1
1.1 Background 2
1.2 Problem Statement 3
1.3 Objectives 3
1.4 Scope 4
1.5 Significance 5
1.6 SDG Alignment 5
1.7 Report Organization 6
Chapter 2: Review of Existing Systems 7
2.1 Overview 9
2.2 Traditional CCTV Surveillance 9
2.3 Consumer Smartwatches and Fitness Wearables 9
2.4 Specialized Industrial Wearables 10
2.5 AI-Based PPE Detection Research 10
2.6 Integrated IoT Safety Platforms 11
2.7 Mobile Safety Applications 11
2.8 Comparative Summary 11
Chapter 3: Proposed System Design 13
3.1 System Overview 14
3.2 Architecture 17
3.2.1 Three-Tier Model 17
3.2.2 Data Flow 18
3.3 Module 1: Vision Intelligence 18
3.3.1 PPE Detection 18
3.3.2 Geofencing and Intrusion Detection 19
3.3.3 Vehicle Proximity Warning 20
3.3.4 Workforce Activity Monitoring 21
3.3.5 Fall Detection 22
3.3.6 Fire and Smoke Detection 23
3.4 Module 2: QR Identification and Supplementary IoT 23
3.4.1 QR-Based Worker Identification 24
3.4.2 Supplementary Smart Vest 25
3.4.3 LoRa Communication Layer 28
3.5 Module 3: Digital Twin Command Center 28
3.5.1 Backend 28
3.5.2 Frontend Dashboard 28
3.6 Module 4: Mobile Application 28
3.6.1 App Architecture 28
3.6.2 Core Features 29
3.6.3 Admin Alert Management 29
3.7 Admin Module 31
3.8 Requirements Specification 31
3.8.1 Interface Requirements 31
3.8.2 Functional Requirements 31
3.8.3 Use Cases 33
3.8.4 Non-Functional Requirements 34
3.8.5 Resource Requirements 35
3.8.6 Database Requirements 35
3.8.7 Project Feasibility 36
3.9 System Workflow 36
3.10 Limitations 38
Chapter 4: Hardware and Software Implementation 39

4.1 Development Environment 41
4.2 QR Identification and Smart Vest 41
4.2.1 QR Worker Identification 41
4.2.2 Smart Vest 42
4.3 PPE Detection Module 43
4.3.1 Dataset and Classes 43
4.3.2 Training Configuration 44
4.3.3 Model Performance 45
4.3.4 Fall Detection 46
4.3.5 Fire and Smoke Detection with Hot-Work Permits 46
4.3.6 Camera Sources and Reliable Event Delivery 46
4.3.7 Vehicle Proximity Detection 47
4.3.8 Danger Areas 47
4.4 Backend Implementation 48
4.5 Frontend Dashboard Implementation 49
4.6 Mobile Application 50
4.6.1 Technology Stack 51
4.6.2 Core Screens 51
4.6.3 Push Notification Flow 52
4.7 Workforce Activity Monitoring 52
4.8 System Architecture and Deployment 53
4.9 SOS and Location Telemetry Flow 56
4.10 End-to-End Alert Flow 57
4.11 Project Timeline 58
Chapter 5: Testing, Results and Discussion 60
5.1 Testing Approach 61
5.2 Test Environment 61
5.3 Automated Test Results 61
5.4 Requirement-Linked Test Cases 62
5.5 Model Evaluation Results 62
5.6 Performance and Reliability Measurements 63
5.7 End-to-End Verification 65
5.8 Defects Found by Testing 65
5.9 Discussion and Threats to Validity 65
Chapter 6: Conclusion and Future Work 66
6.1 Conclusion 67
6.2 Objectives and Outcomes 67
6.3 Limitations 68
6.4 Future Work 68
Appendix I: User Manual 70
Appendix II: Installation and Setup Guide 73
Appendix III: API and Database Reference 75
Appendix IV: Test Evidence 78
References 82

LIST OF FIGURES

Figure 3.1: Use Case Diagram 15
Figure 3.2: System Context Diagram 16
Figure 3.3: Three-Tier System Architecture 17
Figure 3.4: Level-1 Data Flow Diagram 20
Figure 3.5: Workforce Activity Monitoring Pipeline 22
Figure 3.6: Smart Vest Block Diagram 25
Figure 3.7: SOS and Location Alert Flowchart 27
Figure 3.8: Mobile Application Screen Wireframes 30
Figure 3.9: System Workflow Activity Diagram 37
Figure 3.10: Level-0 Data Flow Diagram 38
Figure 4.1: PPE Model Training and Evaluation Pipeline 45
Figure 4.2: Entity Relationship Diagram 49
Figure 4.3: Web Command Centre: Live Camera with a Danger Area, and the Site Map with Cameras 50
Figure 4.4: Mobile Application: Alerts, Site Map, Camera and Alert Detail 52
Figure 4.5: Component View of the Implemented System 54
Figure 4.6: Deployment View 55
Figure 4.7: Camera Pipeline Class Structure 56
Figure 4.8: Vest Telemetry and SOS Sequence 57
Figure 4.9: Camera Event to Dashboard Sequence 58

LIST OF TABLES

Table 2.1: Comparison of Existing Systems/Devices Against Site-Safe.....12
Table 3.1: Worker Identification and Supplementary Sensing.....24
Table 3.2: Mobile App Feature Matrix.....30
Table 3.3: Interface Requirements.....31
Table 3.4: Functional Requirements.....32
Table 3.5: Use Case Descriptions.....33
Table 3.6: Full-Dress Use Case: Detect PPE Violations.....34
Table 3.7: Full-Dress Use Case: Trigger SOS Alert.....34
Table 3.8: Non-Functional Requirements.....35
Table 3.9: Resource Requirements.....35
Table 3.10: Feasibility Assessment.....36
Table 4.1: Development Environment Summary.....41
Table 4.2: Proposed Smart Vest Component Specifications.....42
Table 4.3: Proposed ESP32 Pin Allocation.....43
Table 4.4: Dataset Split Configuration.....44
Table 4.5: YOLOv11 Training Hyperparameters.....44
Table 4.6: Per-Class Detection Performance.....45
Table 4.7: Mobile Application Technology Stack.....51
Table 5.1: Automated Test Suites.....62
Table 5.2: Event-to-Dashboard Latency (NFR-01).....63
Table 5.3: Outage Recovery, 100 Events over a 5-Minute Outage (NFR-02).....64
Table 5.4: Inference Throughput with All Detectors (NFR-04).....64
Table 5.5: Soak Validation Run (NFR-05).....64
Table 6.1: Objectives and Outcomes.....67
Table I.1: Web Command Centre Tasks.....71
Table I.2: Mobile Application Tasks.....72
Table II.1: Setup Steps.....74
Table III.1: Core API Endpoints.....76
Table III.2: Database Schema Summary.....77
Table IV.1: Requirements, Evidence and Status.....79
Table IV.2: End-to-End Verification Record.....80
Table IV.3: Defects Found by Testing.....81

LIST OF ABBREVIATIONS

API Application Programming Interface

ASGI Asynchronous Server Gateway Interface

CI Continuous Integration

DFD Data Flow Diagram

ERD Entity Relationship Diagram

FCM Firebase Cloud Messaging

FPS Frames Per Second

FR Functional Requirement

GPS Global Positioning System

IoT Internet of Things

JWT JSON Web Token

LoRaWAN Long Range Wide Area Network

mAP Mean Average Precision

MQTT Message Queuing Telemetry Transport

NFR Non-Functional Requirement

NMS Non-Maximum Suppression

PPE Personal Protective Equipment

QR Quick Response (code)

REST Representational State Transfer

RTSP Real-Time Streaming Protocol

SDG Sustainable Development Goal

YOLO You Only Look Once

Chapter 1

Introduction

1.1 Background

Walk onto any large construction site in Pakistan and the scene is immediately familiar: dozens of workers moving across multiple floors, heavy machinery operating within close proximity to pedestrians, narrow scaffolding above unprotected drops, and perhaps one or two safety officers trying to keep track of it all. The practical impossibility of this arrangement is obvious to anyone who sees it. What is less obvious is how consistently this arrangement fails, and how predictably those failures result in death or serious injury.

According to the International Labour Organization, construction accounts for roughly 30 to 40 percent of all fatal occupational accidents worldwide despite employing only about 7 percent of the global workforce [16]. In Pakistan, the Labour Force Survey 2020–21 recorded over 3.6 million workers employed in the construction sector, most of whom receive little to no formal safety training and work on sites where PPE compliance is enforced, if at all, through periodic manual checks [17]. Projects funded through initiatives like CPEC have dramatically increased the volume and scale of construction activity without a corresponding investment in safety infrastructure.

The standard approach to construction safety, assigning a safety officer to patrol a site, look for violations, and respond to incidents, has not meaningfully changed in decades. It is a model that was workable when sites were small and crews were compact. On modern sites spanning multiple floors and hundreds of metres, it is not. A single officer physically cannot watch fifty workers simultaneously. Environmental hazards like heat buildup or gas accumulation in excavation zones are invisible to the naked eye. And when something does go wrong, the response is almost always reactive: help arrives after the fall, after the gas exposure, after the equipment strike.

The technological tools to do better than this now exist. Deep learning models can process camera feeds and detect safety violations in real time. Simple printed QR codes can identify individual workers without requiring any electronics on the person. Cross-platform mobile frameworks allow supervisors to receive alerts and monitor workers from anywhere on the site through their phones. When combined, these technologies can create something that has never existed on most Pakistani construction sites: an automated, per-worker compliance record that documents who was wearing what, where, and when. What has been missing is a system that brings all of these capabilities together into a coherent, practical package designed for actual deployment. That is what Site-Safe is intended to be.

1.2 Problem Statement

The specific problem this project addresses is the absence of a continuous, autonomous safety compliance system for construction sites—one that can detect hazards in real time, identify which worker is involved, alert relevant personnel immediately through both web and mobile interfaces, and maintain a verifiable, per-worker record of safety compliance without depending entirely on human attention.

The deficiencies in the current state of the art can be summarized under four headings. First, incomplete coverage: no human supervisor can simultaneously monitor all workers across a multi-floor or multi-zone construction site, and violations happen most frequently precisely when no one is watching. Second, purely reactive response: existing systems only register an incident after it has occurred, with no mechanism for detecting warning signs of an impending accident. Third, no individual accountability: even when a violation is observed, there is no automated way to identify which worker was involved or track repeat offenders across shifts. Fourth, no integrated compliance record: current sites maintain no systematic digital record of PPE compliance, worker activity patterns, or incident history tied to specific individuals, preventing any data-driven safety improvement or auditable compliance documentation over time.

1.3 Objectives

The work carried out under this project is guided by the following specific objectives:

Design and train a real-time PPE detection model based on YOLOv11 that can identify safety equipment across ten detection classes (five items of equipment, each worn and missing) and flag violations under the varied lighting and environmental conditions of an actual construction site.

Implement a QR-based worker identification system where each worker is assigned a unique QR code printed on their helmet, readable by the same cameras running PPE detection, enabling per-worker violation tracking and compliance accountability.

Implement a virtual geofencing system that allows administrators to mark danger zones on a site map, with automatic alerts when workers enter restricted areas, linked to individual worker identity through QR identification.

Develop a web-based command center that aggregates all incoming data and presents it to site managers through an interactive real-time dashboard.

Build a cross-platform mobile application (React Native) that enables safety officers to receive push notifications for critical alerts, view live worker status, and allows administrators to configure and manage alert settings remotely.

Implement workforce activity monitoring through the vision pipeline, tracking worker presence across zones, detecting extended breaks and inactivity periods, and generating per-worker activity logs.

Establish automated incident logging and PDF compliance report generation that documents per-worker violation histories, zone attendance records, and shift summaries, providing construction firms with auditable safety documentation.

Build a lightweight IoT Smart Vest with GPS tracking and an SOS panic button to provide supplementary worker location data and emergency alerts in areas outside camera coverage. The vest's software path is delivered and tested with a simulator; the physical hardware was moved to future work through a scope-change request to the supervisor (Section 4.2.2).

Integrate camera-based fall detection using YOLOv11-Pose, which analyses body keypoints frame-by-frame to identify when a worker has collapsed. A three-second persistence filter reduces false positives from bending or crouching, enabling the system to detect genuine falls and dispatch critical alerts without any wearable sensors.

Deploy a fire and smoke detection model running alongside the PPE pipeline on IP camera feeds. The module includes zone-aware suppression logic so that scheduled hot-work activities such as welding or cutting do not trigger false alarms, while genuine uncontrolled fires still generate critical alerts.

1.4 Scope

This project covers the full-stack design and prototype implementation of Site-Safe. The primary focus is software: the YOLOv11-based vision pipeline with a QR code reader for worker identification, a pose-estimation module for camera-based fall detection, fire and smoke detection governed by hot-work permits, the Django REST backend, the React web dashboard, a cross-platform React Native mobile application for real-time alert management and worker monitoring, and automated compliance reports. The software path of the supplementary Smart Vest (telemetry API, MQTT consumer and a labelled

simulator) is in scope; the physical vest hardware was moved to future work through a scope-change request to the supervisor (Section 4.2.2).

The software scope explicitly includes workforce activity monitoring, compliance documentation, fall detection, and fire safety features. The vision pipeline records when identified workers are on site and when they are idle, producing queryable activity timelines tied to individual worker identities via QR codes. A parallel pose-estimation model monitors body keypoints to detect falls, while a dedicated fire and smoke stage provides early fire warning whose severity is lowered, but never suppressed, during permitted hot-work activities. The mobile application gives supervisors and administrators remote access to alerts and worker status. Automated compliance reports document per-worker violation histories and attendance records.

What is explicitly outside the current scope: the system is designed for outdoor civil construction sites, not enclosed factory floors, mining tunnels, or marine environments. QR code identification requires workers to be within readable range of a camera, so the system works best at key checkpoints and designated work zones rather than across every square metre of a large open site.

1.5 Significance

The relevance of this project goes beyond the technical. Every year, preventable accidents on Pakistani construction sites cut short lives that did not need to be lost. Site-Safe is built on the premise that the technology to prevent many of those accidents already exists; the gap is in deployment, integration, and accessibility.

From a regulatory and commercial standpoint, construction firms face growing legal liability for workplace accidents. A system that automatically logs compliance evidence and generates auditable reports has direct economic value to any firm operating at scale. The mobile application component ensures that safety monitoring is not confined to a control room but extends to every supervisor carrying a smartphone on site.

1.6 SDG Alignment

SDG 3 (Good Health and Well-being): Real-time PPE violation detection, automatic fall detection through pose estimation, and fire and smoke monitoring enable timely intervention before unsafe behaviour leads to injury, directly reducing preventable construction fatalities.

SDG 8 (Decent Work and Economic Growth): Systematically safer working conditions for construction workers, regardless of skill level or contract status, align directly with the decent work agenda. The activity monitoring features add transparency to working hour tracking.

SDG 9 (Industry, Innovation, and Infrastructure): The application of AI, IoT, and mobile technology to modernize construction safety infrastructure is a concrete demonstration of the technology-driven industrial innovation SDG 9 envisions.

SDG 11 (Sustainable Cities and Communities): The fire and smoke detection module, with hot-work permits that distinguish scheduled flame work from uncontrolled fires, contributes to safer urban construction environments, reducing the risk of uncontrolled fires on building sites that endanger both workers and surrounding communities.

1.7 Report Organization

Chapter 2 reviews existing systems and relevant published research, identifying the specific gap that Site-Safe fills. Chapter 3 describes the proposed system design: architecture, module specifications including QR-based identification, fall detection, fire and smoke detection, workforce monitoring, mobile application design, and data flow, followed by the requirements specification, the complete system workflow and the feasibility of the system. Chapter 4 describes the implementation: the development environment, QR identification and the Smart Vest software path, the vision modules, the backend, web and mobile applications, workforce activity monitoring, and the component, deployment, data and interaction views of the delivered system. Chapter 5 reports testing against the requirements traceability matrix, model evaluation and measured performance and reliability. Chapter 6 compares the outcomes with the objectives and sets out limitations and future work. The appendices give a user manual, an installation guide, the API and database reference and the detailed test evidence.

Chapter 2

Review of Existing Systems

2.1 Overview

Before settling on the approach described in Chapter 3, we surveyed the landscape of existing solutions, both commercial products and academic research, that attempt to address construction site safety. What this review revealed, fairly consistently, is that every available solution addresses one piece of the problem while leaving the others untouched. The following sections document what we found.

2.2 Traditional CCTV Surveillance

CCTV remains the most widely used safety monitoring technology in construction environments and has been for decades. Networks of fixed cameras feed into digital recorders accessible via monitor banks or remote connections. In their original form, these systems are entirely passive: they record what happens without analyzing it.

The fundamental limitation of CCTV is not the camera; it is the human being required to watch it. Sustained attention is cognitively demanding, and studies have shown that human observers miss a significant proportion of critical events when monitoring multiple camera feeds over extended periods. Sacks et al. (2022) noted that maintaining basic coverage on a medium-sized construction site required approximately 4.2 full-time monitoring personnel, a cost few firms can justify [6]. Beyond the attention problem, CCTV gives no information about what a camera cannot see: gas concentration in a trench, temperature inside a poorly ventilated enclosure, or how long a worker has been stationary in a hazardous zone. Its role in Site-Safe is not eliminated but redefined: cameras become the front end of an AI inference and activity tracking pipeline, not passive recording devices.

2.3 Consumer Smartwatches and Fitness Wearables

The Apple Watch and comparable devices from Samsung and Garmin have introduced genuinely impressive sensing capabilities into a wrist-worn form factor. Modern smartwatches can detect falls, monitor heart rate, flag irregular cardiac rhythms, and trigger emergency SOS calls. For personal health use, they are excellent.

For construction deployment, they fall short in several practical ways. Bluetooth and Wi-Fi connectivity work well in homes and offices; on a large outdoor site, they become unreliable as soon as a worker moves beyond router range. Battery life of 24 to 48 hours sounds adequate but does not account for continuous sensor polling under the

computational load of real-time monitoring. And devices designed for a commuter are not designed to survive the concrete dust, water exposure, and physical impact of a construction shift. At a retail cost of PKR 60,000 to 150,000 per unit, equipping an entire site crew is financially impractical for the vast majority of Pakistani construction firms.

2.4 Specialized Industrial Wearables

SmartCap Technologies produces a helmet-mounted EEG sensor designed to detect driver fatigue by reading brainwave activity. Within its target application, long-haul trucking and heavy vehicle operation in mining, it performs well. Outside that target, it offers nothing: no PPE compliance detection, no environmental gas sensing, no fall detection capability, and no location or health data beyond fatigue state.

This pattern, high accuracy within a narrow domain and zero applicability outside it, repeats across the industrial wearables category. Proximity warning tags alert workers when machinery approaches but do not monitor heat stress. Heat stress monitors do not detect falls. Fall detectors do not watch for gas. The fragmentation reflects the fact that no existing product was designed with the multi-hazard complexity of a construction site as its primary design constraint.

2.5 AI-Based PPE Detection Research

The academic literature on computer vision applied to construction safety is substantial and growing. The YOLO family of models, introduced by Redmon et al. in 2016, refined in YOLOv3 [1] and developed through successive iterations to YOLOv11 [3], has become the standard architectural choice for real-time PPE detection. The single-pass inference design achieves detection speeds and accuracy levels that are genuinely usable in operational contexts.

Published results are encouraging. Nath, Behzadan, and Paal (2020) achieved 91.2% accuracy on a construction site dataset using YOLOv3 [5]. More recent work with YOLOv8 and YOLOv11 pushes mean Average Precision above 92% on standard benchmarks. The problem is that virtually none of this research moves beyond the dataset. The overwhelming majority of published PPE detection studies evaluate their models on pre-recorded video in controlled settings. They do not deploy on actual edge hardware, integrate with sensor networks, generate real-time alerts, or connect to mobile applications.

They are demonstrations of what is possible in software, not implementations of what works in the field.

2.6 Integrated IoT Safety Platforms

A small number of research prototypes and commercial platforms have attempted to combine sensing modalities for construction safety. Ramli et al. (2020) published a notable IoT-based smart helmet prototype incorporating temperature, gas, and accelerometer sensors with GSM communication [4]. It was a meaningful step, but GSM communication provides only short-range connectivity, the prototype lacks any vision component, and there is no centralized data platform.

Commercial platforms like Pillar Technologies and Triax Technologies offer mesh-networked environmental sensor nodes and BLE worker tags for large construction sites. They provide real data and real utility. They also, notably, do not include computer vision, do not offer long-range communication for large open sites, and are priced for large Western construction enterprises rather than the Pakistani market.

2.7 Mobile Safety Applications

A growing category of construction safety tools focuses on mobile applications. Apps like iAuditor (now SafetyCulture) and Safesite provide digital checklists, incident reporting forms, and basic alert notifications. These tools digitize the paperwork side of safety management and make compliance documentation more convenient.

However, they remain fundamentally manual: a human must observe a violation, open the app, and log it. They do not detect hazards autonomously, do not integrate with IoT sensors or cameras, and do not provide real-time monitoring of worker vitals or location. They are essentially digital clipboards. Site-Safe takes the mobile component further: the mobile app is not just a reporting tool but a real-time monitoring interface connected to an autonomous detection backend, receiving push alerts the instant a hazard is detected and enabling administrators to configure the entire alert system remotely.

2.8 Comparative Summary

Table 2.1: *Comparison of Existing Systems/Devices Against Site-Safe*

System/Device	Year	Vision (AI)	Worker ID / IoT	Long-Range	Mobile App	Dashboard	Primary Limitation
Traditional CCTV	N/A	No	No	No	No	No	Passive, no analytics
Apple/Samsung Watch	2023	No	Partial	No	Yes	No	Short range, expensive
SmartCap (Mining)	2019	No	EEG only	No	No	No	Single-purpose
YOLO Research	2024	Yes	No	No	No	No	Software-only, no deploy
Pillar Technologies	2022	No	Yes	BLE	Partial	Partial	No vision, enterprise price
SafetyCulture App	2023	No	No	N/A	Yes	No	Manual, no detection
Ramli et al.	2020	No	Yes	GSM	No	No	No vision, no platform
Site-Safe (ours)	2026	Yes	QR (+ GPS vest, proposed)	LoRa (proposed)	Yes	Yes	Prototype; QR range not yet measured

The table above makes the gap plain. No existing system, commercial or academic, combines computer vision-based PPE detection with QR-based individual worker identification, workforce activity monitoring, automated compliance documentation, a centralized web dashboard, and a mobile application for real-time alerts and administration. Site-Safe is designed to be that system.

Chapter 3

Proposed System Design

3.1 System Overview

Site-Safe is structured around three distinct layers that work together: edge devices on the construction site that capture and process visual data, a network layer that moves that data reliably from site to server, and an application layer that stores, processes, displays information through a web dashboard, and pushes critical alerts to a mobile application. The design deliberately separates these concerns so that individual components can be upgraded or replaced without requiring changes to the whole system.

On the site itself, the primary edge devices are AI cameras running YOLOv11 inference for both PPE detection and QR-based worker identification. Each worker wears a helmet with a unique printed QR code, which the same camera reads to establish identity without requiring any electronics on the person. A lightweight Smart Vest with GPS and SOS capability is proposed for supplementary tracking in areas outside camera coverage (Section 4.2.2). Everything converges at the backend server, which processes incoming data, evaluates alert rules, generates compliance records, and pushes updates to both the web dashboard and the mobile app in real time.

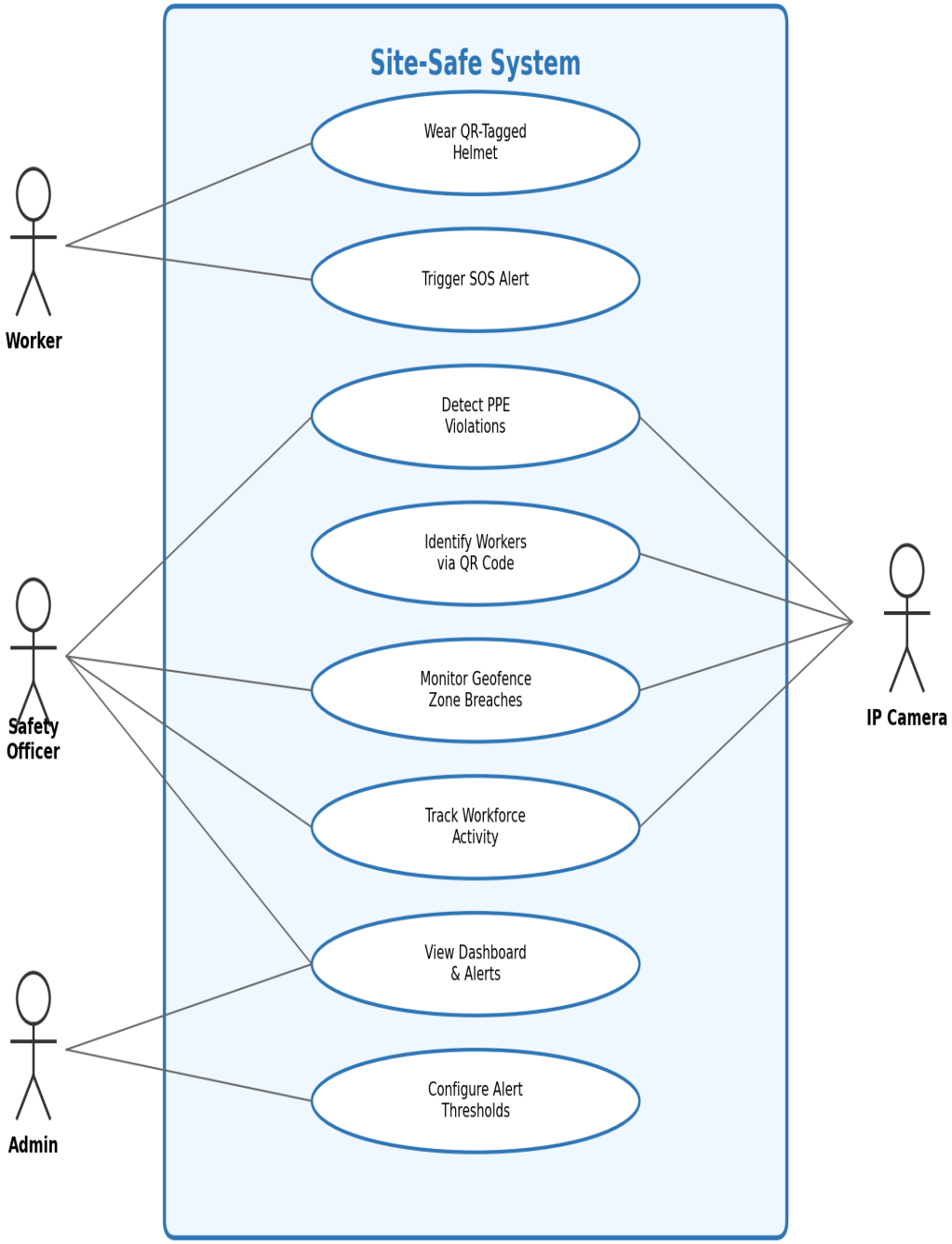


Figure 3.1: *Use Case Diagram*



Figure 3.2: *System Context Diagram*

3.2 Architecture

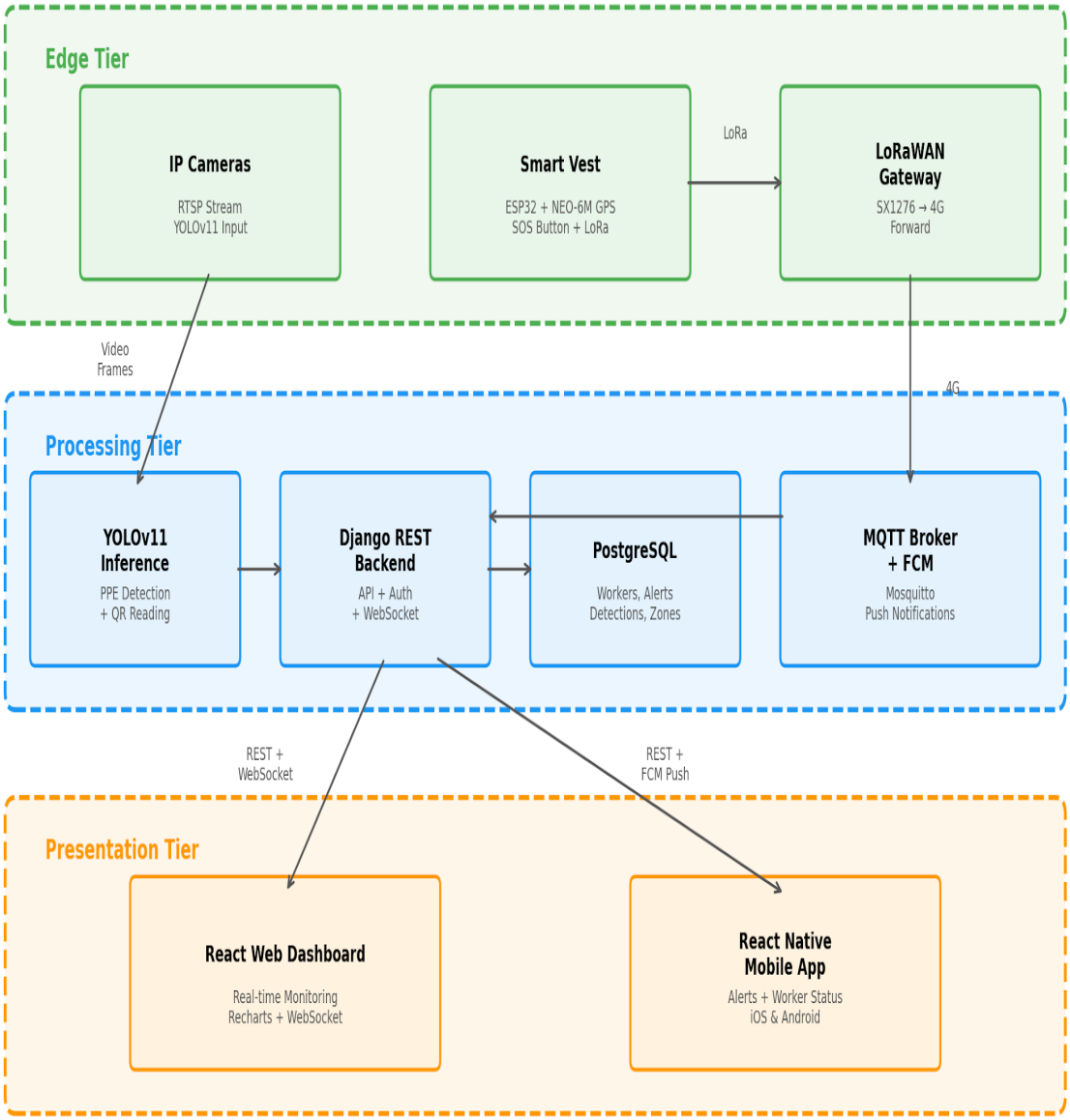


Figure 3.3: *Three-Tier System Architecture*

3.2.1 Three-Tier Model

Tier 1 (Edge): IP cameras feed an edge computer that runs a multi-model inference pipeline locally. A Raspberry Pi 5 or Jetson-class unit was proposed; the prototype runs on a laptop GPU (Section 4.1). The primary YOLOv11 model handles PPE detection on every frame; a person-and-vehicle detector and QR code reading run on every third frame, a YOLOv11-

Pose model runs on every third frame for fall detection, and a fire and smoke model runs on every fifth frame. Together these models push violation alerts, fall events, fire detections and worker identification data to the backend, and the same pipeline feeds workforce activity monitoring. A lightweight ESP32-based Smart Vest is proposed to provide supplementary GPS location and SOS alerts via LoRa.

Tier 2 (Network): Camera alerts and QR identification data are sent over HTTP to the backend API, authenticated with per-camera device tokens and buffered in a local outbox while the server is unreachable. For the supplementary Smart Vest, a LoRa receiver is proposed to forward GPS and SOS data via MQTT; an MQTT broker (Mosquitto) handles the publish-subscribe routing, and the vest gateway can also post over HTTP.

Tier 3 (Application): A Django REST backend processes all incoming data, applies business rules (threshold comparisons, geofence polygon checks, activity tracking logic), and persists results to PostgreSQL. Real-time updates are pushed to the React.js web dashboard via Django Channels WebSockets, and critical alerts are forwarded to the React Native mobile app through Firebase Cloud Messaging push notifications.

3.2.2 Data Flow

Data moves through the tiers in three flows. Camera frames are analysed on the edge, and only confirmed events, with the worker's badge and the evidence frame, are posted to the API. Vest positions and SOS presses reach the API over HTTP or MQTT. Configuration (zones, cameras, alert rules and permits) flows from the dashboard to the database, and cameras poll their rules from it. Figure 3.4 shows these flows as a Level-1 data flow diagram; Section 3.9 gives the Level-0 diagram and the complete workflow.

3.3 Module 1: Vision Intelligence

3.3.1 PPE Detection

The vision module uses a YOLOv11-M model to continuously inspect camera frames for ten detection classes covering helmets, high-visibility vests, boots, gloves and goggles, each in a worn and a missing state. Frames are captured from IP cameras over RTSP or HTTP streams, resized to 640×640 and passed through the model on the edge device. Detections below a candidate confidence threshold (0.15 in the prototype, pending per-class calibration) are discarded. OpenCV’s QRCodeDetector scans sampled frames for QR

codes on worker helmets, linking a detected person to their registered identity. A worker without required PPE triggers a violation event logged with worker identity (from QR, when readable), timestamp, camera ID, zone and detection confidence, creating an auditable per-worker compliance record.

3.3.2 Geofencing and Intrusion Detection

Administrators define Red Zones by drawing polygon boundaries directly on the site map in the dashboard, and the polygons are stored in the database. A camera that reads a worker's QR badge places the worker in that camera's zone, and vest GPS positions are checked against the zone polygons with a point-in-polygon test. An incursion into a restricted zone triggers an immediate alert on the dashboard and a push notification to the safety officer through the mobile app. Hazards smaller than a zone are drawn as danger areas on a camera's picture (Section 4.3.8).

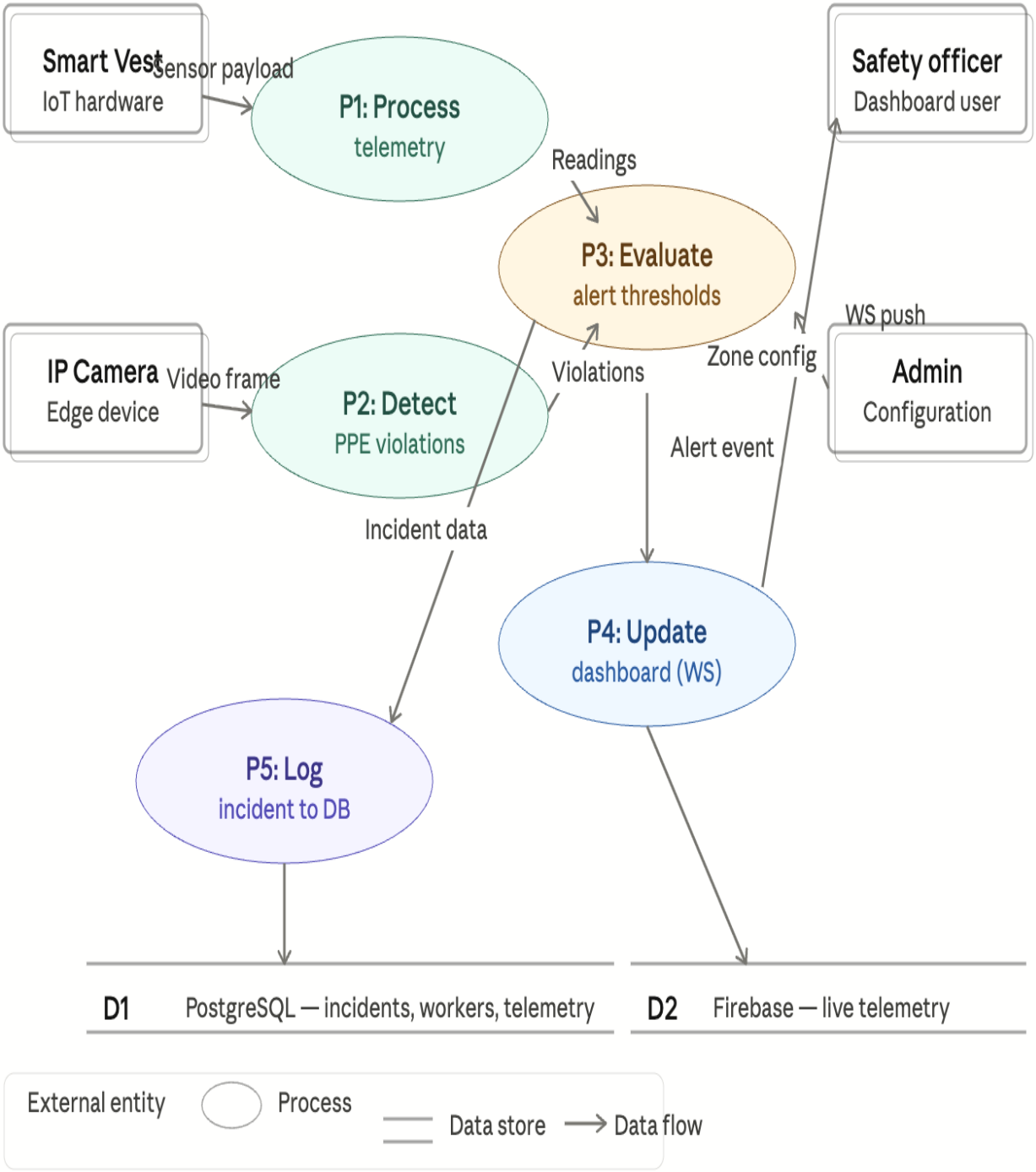


Figure 3.4: *Level-1 Data Flow Diagram*

3.3.3 Vehicle Proximity Warning

A parallel detection class identifies heavy construction vehicles: cranes, dump trucks, forklifts, bulldozers. When a vehicle bounding box and a person bounding box appear within a defined proximity threshold, a proximity warning is generated for that camera zone. This catches the blind-spot hazard that accounts for a meaningful proportion of construction vehicle accidents.

3.3.4 Workforce Activity Monitoring

Beyond safety compliance, the vision pipeline is extended to serve as a workforce activity monitoring layer. Using the same YOLOv11 person detection output, the system tracks three key metrics. Activity tracking logs when each detected worker enters and exits designated work zones, producing timestamped records of zone presence duration. Break detection identifies workers who remain stationary beyond a configurable threshold (default: 15 minutes) or who leave their assigned zone for extended periods, flagging these as break events. Inactivity monitoring raises an alert when a worker shows no detectable movement for a prolonged duration, which may indicate injury, distress, or a safety concern requiring immediate investigation.

All activity data is stored in PostgreSQL with worker ID, zone ID, and timestamp fields, making it queryable by date range, worker, or zone. This data feeds into both the web dashboard (where it appears as activity charts and zone utilization heat maps) and the mobile app (where supervisors can check any worker's current status and recent activity history).

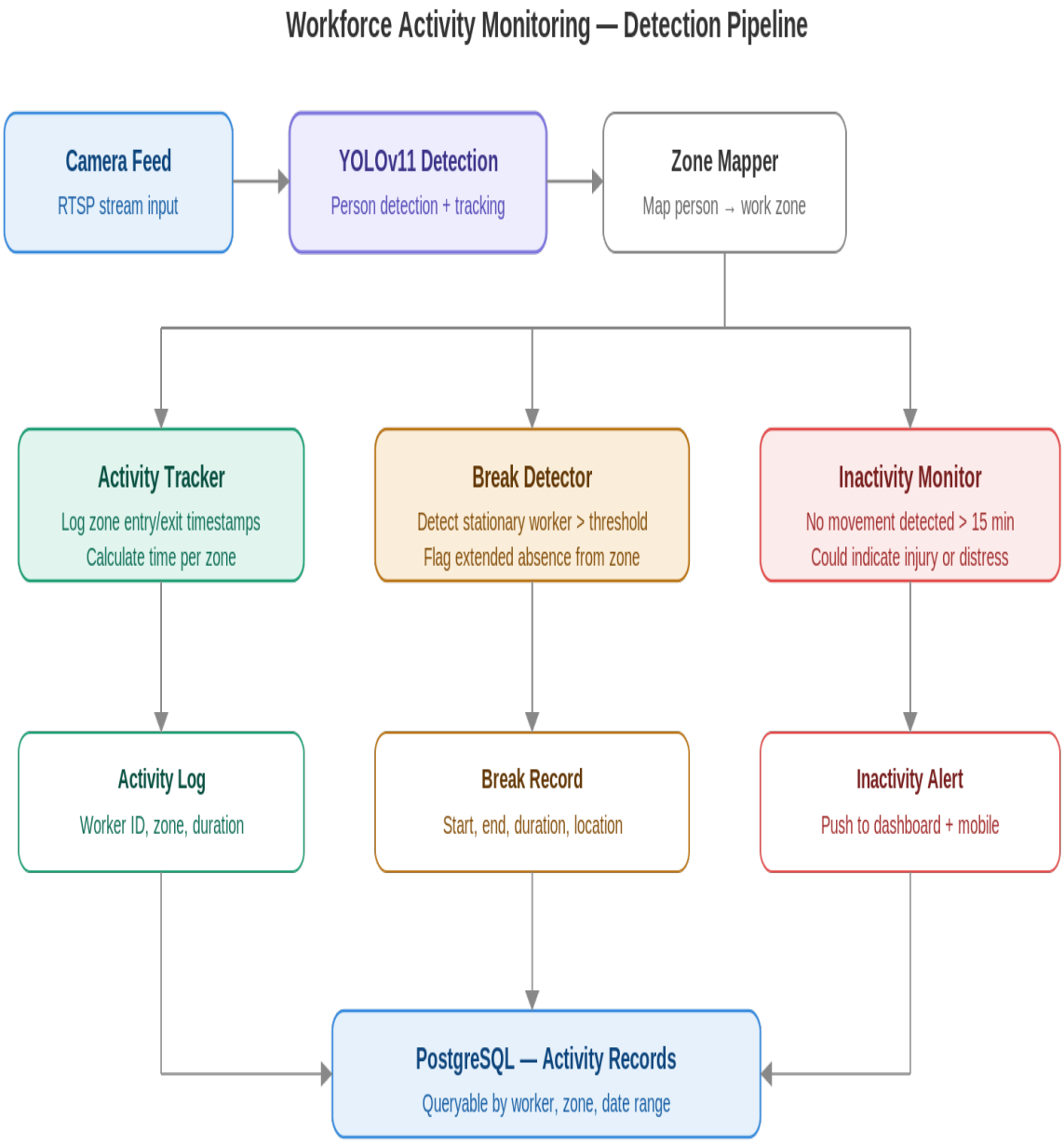


Figure 3.5: *Workforce Activity Monitoring Pipeline*

3.3.5 Fall Detection

A second inference model, YOLOv11-Pose (6.4M parameters), runs alongside the PPE detector on every third frame to conserve compute. The pose model outputs 17 body keypoints per detected person: nose, eyes, ears, shoulders, elbows, wrists, hips, knees, and ankles. Fall detection relies on two geometric signals derived from these keypoints. First, the system measures the ratio of a person's horizontal span to their vertical span; a standing

person is taller than wide, while a fallen person is wider than tall. Second, it checks whether the hip keypoints have dropped to or below knee level, indicating the torso has collapsed.

To reduce false positives from workers crouching, bending to pick up materials, or sitting during breaks, a three-second persistence filter is applied. A fall event is only raised if the collapsed posture persists for at least three consecutive seconds. This filters out the brief postural changes that are normal on a construction site. When a genuine fall is confirmed, the system generates a CRITICAL alert with the detection confidence score, duration, and a bounding box snapshot of the frame, dispatched instantly through WebSocket and FCM push notifications.

3.3.6 Fire and Smoke Detection

A dedicated fire and smoke detection model runs on every fifth frame from the same IP camera feeds. The model is a YOLO variant fine-tuned on construction-environment fire and smoke datasets, outputting two classes: fire and smoke. Each detection includes a bounding box, confidence score, and estimated region area in pixels.

A key challenge on construction sites is distinguishing between controlled fire activities (welding, cutting, grinding) and genuine uncontrolled fires. Site-Safe addresses this through zone-aware suppression. Each zone in the system can be tagged with permitted activities such as welding or hot work. Additionally, safety officers can issue time-bound hot work permits through the dashboard, specifying a zone, start time, and end time. When the fire model detects flame or smoke in a zone with an active hot work permit, the alert is either suppressed entirely or downgraded to LOW severity and logged without triggering a push notification.

A secondary heuristic tracks whether the detected fire region is growing across consecutive frames. A welding torch produces a small, steady flame that does not expand, while an uncontrolled fire grows over time. If the bounding box area increases by more than 50 percent over a rolling window of 30 frames, the system treats it as a genuine fire regardless of any active hot work permit, escalating the alert to CRITICAL with full push notification dispatch.

3.4 Module 2: QR Identification and Supplementary IoT

3.4.1 QR-Based Worker Identification

The primary method of identifying individual workers on site is through QR codes printed on their helmets. During registration, the admin assigns each worker a unique QR code generated by the system. The QR is printed as a weatherproof sticker and attached to the top or side of the worker's helmet, visible to overhead and angled cameras. OpenCV's built-in QRCodeDetector runs on each frame alongside YOLOv11, decoding any visible QR and mapping it to the worker's database record. This approach requires no electronics on the worker, costs virtually nothing per unit, and survives the dust, water, and impact conditions of a construction site. The trade-off is that QR readability depends on camera distance and angle, so the system works best at designated checkpoints and work zones where cameras are positioned within 5–10 metres.

Table 3.1: *Worker Identification and Supplementary Sensing*

Component	Model	Function	Interface	Sample Rate
GPS	NEO-6M	Latitude, longitude	UART	1 Hz
Panic Button	Tactile Switch	Manual SOS	GPIO Interrupt	Event-driven

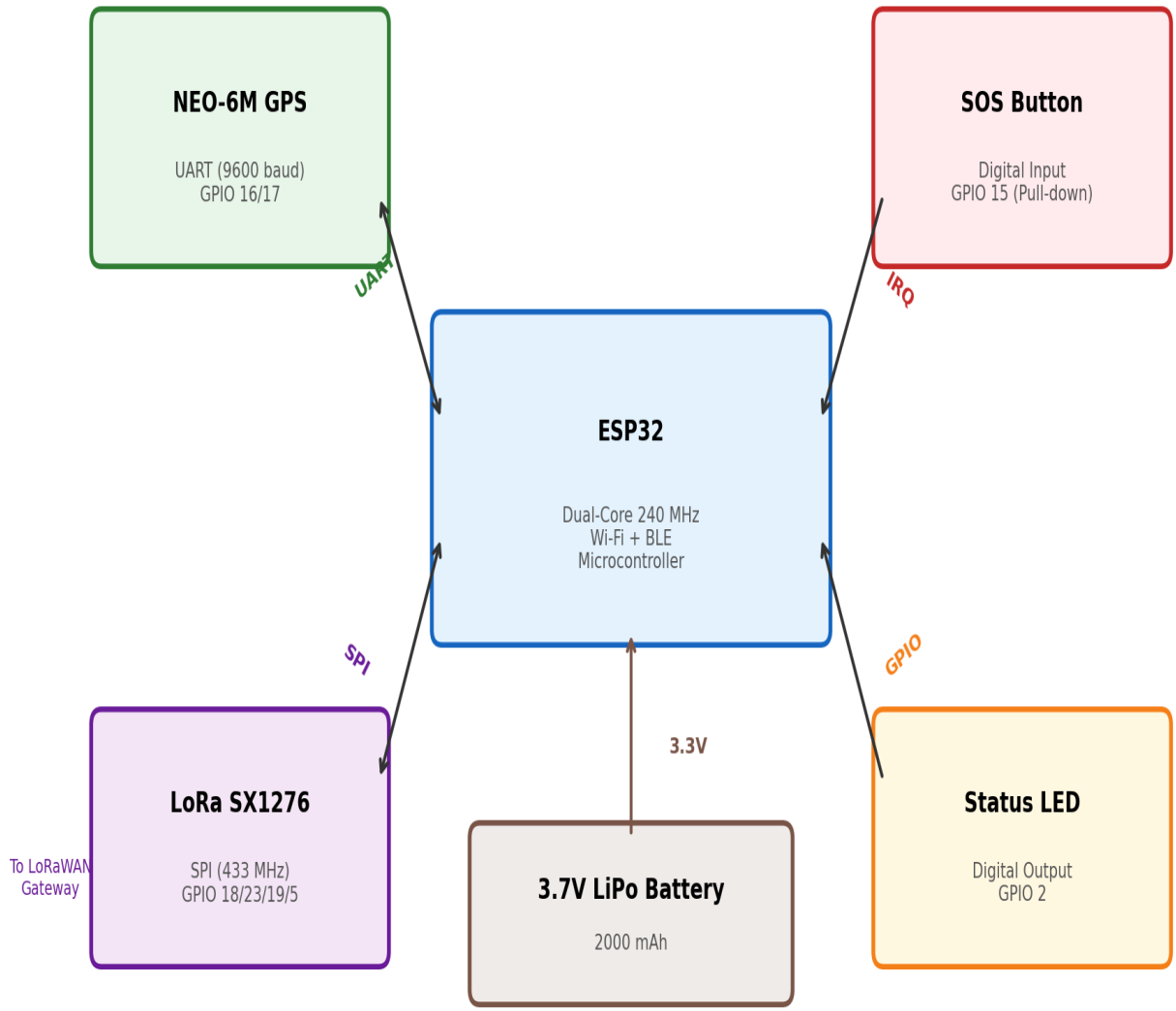


Figure 3.6: *Smart Vest Block Diagram*

Power is supplied by a 3.7V 2500mAh LiPo battery managed through a TP4056 charge controller with overdischarge protection. With only GPS and LoRa active at 30-second transmit intervals, estimated battery life exceeds 24 hours, comfortably covering a full construction shift without the power drain of multi-sensor sampling.

3.4.2 Supplementary Smart Vest

For areas outside camera coverage, a lightweight Smart Vest built around the ESP32 microcontroller provides supplementary worker tracking. The vest is deliberately minimal: a GPS module (NEO-6M) for location tracking and a tactile SOS panic button for emergencies.

This design reflects the project's camera-first philosophy—the vest exists to fill gaps, not to serve as the primary monitoring layer. The SOS button triggers an immediate high-priority alert through all channels when pressed, and the GPS module transmits the worker's coordinates at configurable intervals so the system can track location in zones without camera coverage.

By keeping the vest hardware minimal, we avoid the cost, complexity, and battery-life problems that plague multi-sensor wearables in construction environments. A GPS module and a button are simple to manufacture, easy to maintain, and require no worker training beyond "press this if you need help." The MQTT communication layer means the system architecture can accept additional sensors in future iterations without any backend changes.

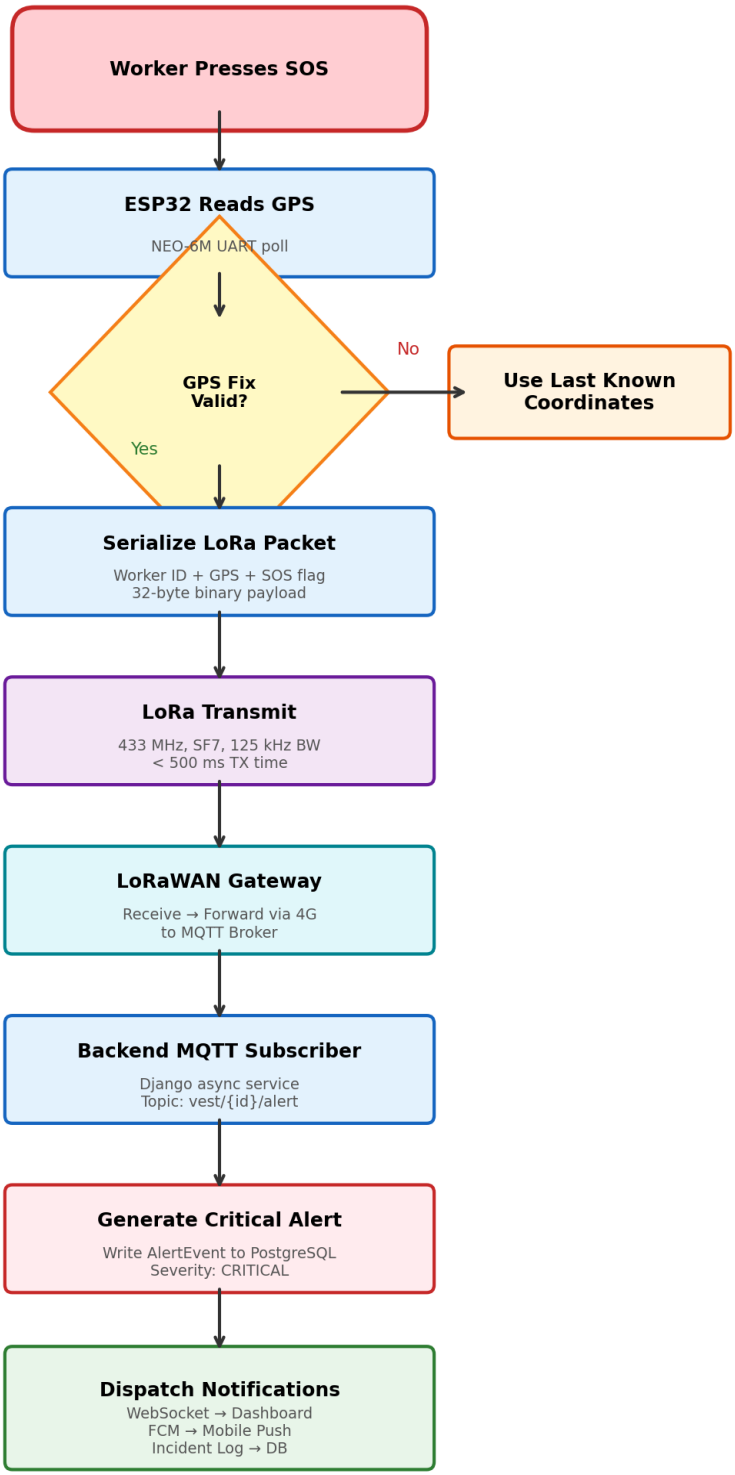


Figure 3.7: *SOS and Location Alert Flowchart*

3.4.3 LoRa Communication Layer

The vest transmits GPS coordinates and SOS alerts over LoRa at 433 MHz using an RA-02 transceiver module. LoRa was selected because it achieves 2 to 15 km range in open terrain at very low power consumption, which suits large construction sites where Wi-Fi infrastructure is impractical. Data is serialized into a compact binary packet and transmitted every 30 seconds under normal conditions, with immediate transmission triggered by an SOS button press.

3.5 Module 3: Digital Twin Command Center

3.5.1 Backend

The Django REST Framework backend provides the API layer that edge devices write to and both the dashboard and mobile app read from. A separate MQTT consumer process handles incoming vest telemetry, and critical alerts are routed to WebSocket clients and to the FCM push notification service. Notifications are sent from a background thread pool after each alert is committed, and PDF and DOCX reports are produced by a separate Node.js report service, so neither blocks API response times.

3.5.2 Frontend Dashboard

The React.js dashboard is organized around a central site view with worker avatars at their current GPS-derived positions, color-coded by status: green for normal, yellow for a warning condition, red for an active alert. Surrounding the main view are panel components for the live alert feed, worker status cards, activity monitoring charts, and incident history. All data updates arrive over WebSocket and trigger targeted component re-renders rather than full page refreshes.

3.6 Module 4: Mobile Application

3.6.1 App Architecture

The mobile application is built with React Native, enabling a single JavaScript codebase to target both iOS and Android platforms. React Native was chosen specifically because the

team already has React expertise from the web dashboard development, and the component model and API service patterns carry over between web and mobile.

The app communicates with the same Django REST backend used by the web dashboard. Authentication uses JWT tokens stored securely in the device keychain. Real-time alert delivery uses Firebase Cloud Messaging (FCM), which provides reliable push notifications even when the app is backgrounded or the device is locked. For live data when the app is in the foreground, a WebSocket connection provides the same real-time stream as the web dashboard.

3.6.2 Core Features

The mobile app provides four primary capabilities. First, a live alert feed showing all active alerts with severity color-coding, source identification (camera or vest), and timestamp, with one-tap navigation to detailed alert views. Second, a worker status screen showing all registered workers with their current location zone, compliance status, and status badge (active, warning, SOS, offline). Third, push notifications for critical alerts (PPE violations, SOS, inactivity, zone breaches) that function even when the app is not in the foreground. Fourth, an activity summary view where supervisors can check any worker's zone history, violation record, time worked, and break periods for the current shift.

3.6.3 Admin Alert Management

For users with Admin role privileges, the mobile app exposes an alert configuration interface. Admins can enable or disable specific alert categories (PPE violations, fall detection, fire and smoke, SOS alerts, zone breaches, inactivity alerts), set custom threshold values for each category (inactivity timeout, zone boundary parameters, fall persistence duration), manage hot work permits for zones where welding or cutting is scheduled, configure notification routing (which roles receive which alert types), and set shift schedules that determine when monitoring is active. All configuration changes sync immediately to the backend and take effect across the entire system.

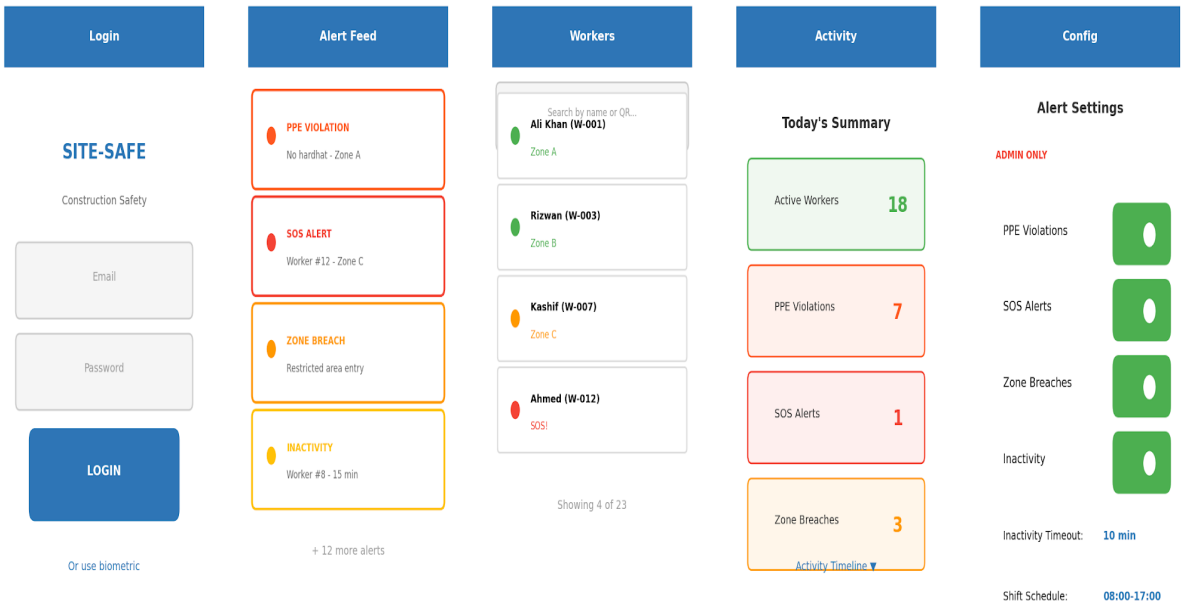


Figure 3.8: Mobile Application Screen Wireframes

Table 3.2: *Mobile App Feature Matrix*

Feature	**Safety Officer**	**Admin**	**Description**
Live alert feed	Yes	Yes	Real-time list of all active alerts with severity and source
Worker status view	Yes	Yes	Current location, compliance status, and alerts for all workers
Push notifications	Yes	Yes	FCM alerts for SOS, PPE violations, zone breaches
Activity summary	Yes	Yes	Zone history, time worked, break periods per worker
Enable/disable alerts	No	Yes	Toggle specific alert categories on/off
Set thresholds	No	Yes	Configure inactivity timeout, zone boundaries, alert routing
Notification routing	No	Yes	Assign alert types to specific roles
Shift scheduling	No	Yes	Define active monitoring windows

3.7 Admin Module

The admin interface, accessible through both the web dashboard and the mobile app, handles site configuration and worker management. Admins can register workers and assign QR codes to their profiles, draw geofence zones on the site map, configure alert and notification preferences, and generate or download PDF compliance reports for any date range. Access is role-gated: Admin, Safety Officer, and read-only Viewer roles are defined with corresponding permission sets enforced at the API level.

3.8 Requirements Specification

3.8.1 Interface Requirements

Table 3.3 lists the interfaces between the components and with external systems. Cameras and vest gateways never sign in as users; each authenticates with a device token limited to its own kind of data.

Table 3.3: Interface Requirements

Interface	Type	Requirement
Camera to edge computer	Hardware	IP camera over RTSP or HTTP, or a USB camera, placed so badges (60 mm) are readable; tested at 1280×720
Edge computer	Hardware	GPU able to run all detectors at 10 FPS or more (NFR-04)
Vest to gateway	Hardware	ESP32 with GPS (UART) and SOS button (GPIO); LoRa link to a gateway (proposed, Table 4.2)
Camera and gateway to API	Software	JSON over HTTP (HTTPS on a site network) with a device token, Authorization: Device <key>, one per camera or gateway
Gateway to broker	Software	MQTT topics sitesafe/vest/<id>/telemetry and /sos on Mosquitto
Clients to API	Software	REST with a 30-minute JWT renewed by a refresh token; WebSocket /ws/dashboard/ with the same token
Backend to services	Software	PostgreSQL 14 or later; Redis channel layer; Node.js report service; Firebase Cloud Messaging and SMTP
Users	Software	Current desktop browser for the command centre; Android or iOS phone (Expo) for the mobile app

3.8.2 Functional Requirements

Table 3.4 lists the functional requirements with their priority (M must, S should) and the acceptance criterion each is tested against in Chapter 5. The criteria are the team's proposals.

Table 3.4: Functional Requirements

ID	Requirement	Pri.	Acceptance criterion
FR-01	Sign in with e-mail and password; sessions renew silently	M	Expired access token renews; revoked refresh token returns to sign-in
FR-02	Admin, Safety Officer and Viewer roles enforced by the API	M	Every forbidden action returns 403; its permitted counterpart succeeds
FR-03	Interface offers only actions the role permits	S	Viewer sees no resolve, edit or generate controls
FR-04	Detect missing PPE (10 classes) from a camera	M	Event precision ≥ 90% and episode recall ≥ 85% on a sealed target-camera set
FR-05	One alert per person per violation episode	M	Two people give two alerts; a one-frame flicker gives none
FR-06	Detect falls from camera pose	M	Fall and hard-negative videos meet the AI event targets
FR-07	Warn when a worker is too close to a vehicle	M	One alert per person per encounter; threshold evaluated on video
FR-08	Detect prolonged inactivity with a dashboard-set timeout	M	A changed rule changes the camera timeout within 60 s
FR-09	Fire and smoke detection governed by hot-work permits	M	Fixture videos pass; a permitted fire is downgraded, never dropped
FR-10	Identify workers by the QR badge on the helmet	M	Association, unknown and wrong-person rates measured at stated distance and light
FR-11	Camera events reach the API reliably	M	100 events queued in a 5-minute outage arrive exactly once
FR-12	Only authorised devices submit data	M	Anonymous, revoked, wrong-kind or wrong-camera tokens are rejected
FR-13	Live updates for every alert, vest and worker change	M	Two sessions converge after create, resolve, SOS and reconnect
FR-14	WebSocket requires a valid user token and allowed origin	M	Missing, bad or expired token closes with code 4401
FR-15	Alert rules (enable, threshold, notify) change behaviour	M	A disabled type stores nothing; SOS and FIRE cannot be disabled
FR-16	Push notifications to subscribed users	S	An Android phone receives a push; tapping opens the alert
FR-17	E-mail notifications	S	Mail reaches opted-in users when the rule enables it
FR-18	Register, edit and deactivate workers in the interface	M	All three work in the browser; one vest per worker
FR-19	Zones with polygon boundaries and assigned cameras	M	Invalid geometry rejected; boundary drawn on a map
FR-20	Automatic restricted-zone breach and over-occupancy alerts	M	One ZONE_BREACH per unassigned entry or excess occupancy
FR-21	Smart Vest: GPS position, SOS button, LoRa link	M	A physical SOS reaches the dashboard (hardware moved to future work)
FR-22	Vest telemetry is validated	M	Out-of-range coordinates or battery return 400
FR-23	Vest offline detection	S	A vest silent for 120 s is shown offline
FR-24	Activity history: check-in, check-out, idle	M	Recorded automatically and reconciled with annotated scenarios
FR-25	Incident list, detail, resolve and CSV export	M	Resolution is broadcast; filters cover every alert type
FR-26	Dashboard shows recorded data only	M	Every figure reconciles to the API
FR-27	Incident and safety-performance reports in DOCX and PDF	M	Fixtures reconcile exactly; no unsupported percentages
FR-28	Mobile app: alerts, resolve, workers, site map, cameras, push	M	An Android build receives a push and resolves an alert seen on the web
FR-29	Alert evidence: snapshot, confidence, model version	S	Each AI alert is traceable to its frame and model

ID	Requirement	Pri.	Acceptance criterion
FR-30	Live camera grid for monitoring	S	Running cameras appear live to Safety Officers and Admins within seconds; Viewers see no video
FR-31	Locate workers and count people per zone by camera	S	A badge read places the worker in the camera's zone; overlapping cameras never double-count
FR-32	Danger areas drawn on the camera picture	S	A person inside an active area for 1 s raises one alert; walking past raises none
FR-33	Cameras on the site map	S	Placed cameras show their coverage and live, offline or alerting state
FR-34	Evidence snapshots in reports	S	Reports show captioned frames of the most severe incidents and state what is missing

3.8.3 Use Cases

Table 3.5 describes the use cases of Figure 3.1. Two are written as full-dress use cases because they cross several components and have failure paths: detecting a PPE violation (Table 3.6), which every camera hazard follows, and triggering an SOS (Table 3.7).

Table 3.5: Use Case Descriptions

Use case (Figure 3.1)	Actors	Description	FRs
UC-1 Wear QR-tagged helmet	Worker	Wears the printed badge so cameras can attribute events	FR-10
UC-2 Trigger SOS alert	Worker	Presses the vest SOS button (Table 3.7)	FR-20–23
UC-3 Detect PPE violations	IP Camera, Safety Officer	Camera raises a violation; officer responds (Table 3.6)	FR-04, 05, 29
UC-4 Identify workers via QR	IP Camera	Badge decoded and linked to the person's box	FR-10
UC-5 Monitor zone breaches	Safety Officer, Admin	Restricted entry, over-occupancy or a danger-area entry raises an alert	FR-19, 20, 31, 32
UC-6 Track workforce activity	Safety Officer	Attendance, idle time and timelines per worker	FR-08, 24
UC-7 View dashboard and alerts	Safety Officer, Admin	Live alerts, cameras, site map, incidents and reports on web and mobile	FR-13, 25–28, 30, 33, 34
UC-8 Configure alert thresholds	Admin	Enable rules, set thresholds and notifications	FR-15

Table 3.6: Full-Dress Use Case: Detect PPE Violations

Field	UC-3 Detect PPE Violations
Scope and level	Site-Safe system; user goal
Primary actor	IP Camera (initiates); Safety Officer (responds)
Stakeholders	Safety Officer: a prompt alert naming the worker. Worker: no false accusation, images kept no longer than needed. Management: an auditable record.
Preconditions	The camera has a device token and is assigned to a zone; the PPE rule is enabled; the officer is signed in.
Success guarantee	One alert per person per episode is stored with type, severity, camera, zone, worker if identified, confidence, model version and snapshot, shown live, and its resolution recorded.
Main success scenario	1. The pipeline detects a missing-PPE box on a person. 2. The episode persists for 1 s in at least 3 frames. 3. The person's badge is read and the evidence attached. 4. The event is queued and posted. 5. The server checks the token and rule and stores the alert once. 6. Dashboard and app show it; push and e-mail follow the rule. 7. The officer reviews the snapshot, acts and resolves it. 8. Every session shows it resolved.
Extensions	2a. Shorter flicker: no alert. 3a. Badge unreadable or ambiguous: the alert is stored as not identified. 4a. Server unreachable: the event stays in the outbox and is retried until stored once. 5a. Rule disabled: reply 202, nothing stored. Fall, proximity, inactivity and fire follow the same path with their own confirmation; fire severity depends on hot-work permits.
Special requirements	NFR-01 latency, NFR-02 outage tolerance, NFR-03 security; snapshots of at most 512 KB, deleted after 30 days.

Table 3.7: Full-Dress Use Case: Trigger SOS Alert

Field	UC-2 Trigger SOS Alert
Scope and level	Site-Safe system; user goal
Primary actor	Worker; supporting actor: vest gateway
Stakeholders	Worker: help arrives quickly at the right place. Safety Officer: one clear alert with a position.
Preconditions	The vest is registered and assigned to the worker; the gateway has a device token.
Success guarantee	One CRITICAL SOS alert with the worker and position is stored, shown live and notified; the worker is checked in.
Main success scenario	1. The worker presses SOS. 2. The vest sends position and SOS state through the gateway (HTTP or MQTT). 3. The server validates it, updates the position and finds the zone. 4. On the rising edge of SOS it raises the alert. 5. The map and app show the alert and location; notifications are sent. 6. The officer responds and resolves it.
Extensions	1a. Button held: no repeat alert. 3a. Invalid payload: rejected with 400. 3b. Restricted zone: a ZONE_BREACH is also raised. Vest silent for 120 s: shown offline. Simulator token: alert labelled SIMULATED and excluded from reports.
Special requirements	Physical vest moved to future work; verified with the simulator (FR-21).

3.8.4 Non-Functional Requirements

Table 3.8 lists the non-functional requirements; Section 5.6 reports their measurement.

Table 3.8: Non-Functional Requirements

ID	Category	Requirement	Target (proposed)
NFR-01	Performance	Event-to-dashboard latency	95th percentile ≤ 2 s on a local network
NFR-02	Reliability	Outage tolerance	No lost or duplicated events across a 5-minute API outage and sender restart
NFR-03	Security	Producer security	No anonymous write endpoint; no credentials in source code
NFR-04	Performance	Inference throughput	≥ 10 FPS with all detectors on the demonstration machine
NFR-05	Reliability	Soak	2-hour full-stack run with a controlled disconnect
NFR-06	Portability	Reproducible setup	A fresh machine reaches a working demonstration from the README
NFR-07	Maintainability	Quality gate	All test suites pass; frontend builds and lints clean

3.8.5 Resource Requirements

Table 3.9 lists the resources used to build and run the prototype. Costs are not tabulated: the software is open source, and quotations for the vest parts have not yet been recorded.

Table 3.9: Resource Requirements

Resource	Used in the prototype	Justification
Development and edge computer	Apple M4 laptop, 16 GB, built-in GPU	Runs all detectors at 15.7 FPS (NFR-04); a Jetson-class unit is the site option
Training GPU	Google Colab and Kaggle, Tesla T4	Training exceeded the laptop's memory; the fire model was trained on Kaggle
Cameras	Webcam, phone IP-camera application and recorded test videos	Same code path as site RTSP cameras
Phone	Android phone (push test pending)	Firebase push needs a development build on a device
Smart Vest parts	Not bought; simulator used	ESP32, GPS, LoRa and button of Table 4.2 when the vest is built
Software and data	Open-source stack (Table 4.1); CC BY 4.0 dataset	No licence cost; dataset licence permits reuse with attribution
Human effort	Three members over two semesters	[Task division and person-months to be completed by the team]

3.8.6 Database Requirements

The system needs a relational database (PostgreSQL) that stores each camera event exactly once under its event identifier, keeps every alert with its worker, zone, permit, model version and snapshot, records attendance at the time of the event so that late deliveries merge into their session, and holds only a hash of each device key. Snapshots are deleted after the retention period while their alerts are kept. Figure 4.2 shows the data model and Appendix III the tables.

3.8.7 Project Feasibility

Table 3.10 assesses the feasibility of the system, including its cost, health and safety, legal, ethical, societal and environmental aspects.

Table 3.10: Feasibility Assessment

Aspect	Assessment
Technical	Feasible. Every component uses mature open-source frameworks, the full detector set runs at 16 FPS on a laptop GPU and 470 automated tests pass. The open risk is detection of missing equipment (Section 5.5).
Operational	Feasible. Officers keep their role and receive alerts on their phones; workers need only a paper badge; zones are drawn on a map. Cameras must be placed so badges are readable.
Economic	Low cost. The software is open source, existing IP cameras can be reused and badges cost a sheet of paper; the main expense is an edge GPU computer per site, plus optional vest hardware.
Legal and ethical	Camera monitoring needs informed worker consent and signage. Badges carry a random code, not personal data; snapshots are visible only to signed-in users, deleted after 30 days and can be switched off. The system informs the officer and makes no disciplinary decision.
Health, safety and society	Aims to prevent injuries and deaths (SDG 3 and 8). Missed detections remain possible, so it supplements supervision rather than replacing it.
Environmental	Runs on existing hardware; the only consumables are paper badges and, later, vest batteries.

3.9 System Workflow

Figure 3.9 shows the complete workflow as an activity diagram, from setting up a site to the compliance report. An administrator registers workers and prints their QR badges, draws zones, assigns cameras and sets alert rules, and safety officers issue hot-work permits for planned flame work. While the site runs, each camera pipeline captures frames and runs its detectors; an unconfirmed hazard returns the loop to the next frame. A confirmed hazard is tagged with the person's badge, the confidence, the model version and a snapshot, queued in the outbox and posted with the camera's device token.

The backend authenticates the device, discards alert types whose rule is disabled, sets the severity, which for fire and smoke depends on any active permit, and stores the alert once. After the transaction commits, the alert appears live on the dashboard and the mobile app and notifications are sent. The officer reviews the snapshot, acts on site and resolves the alert, and the stored records feed the attendance timelines and reports. Vest telemetry joins the same path at validation (Section 4.9).

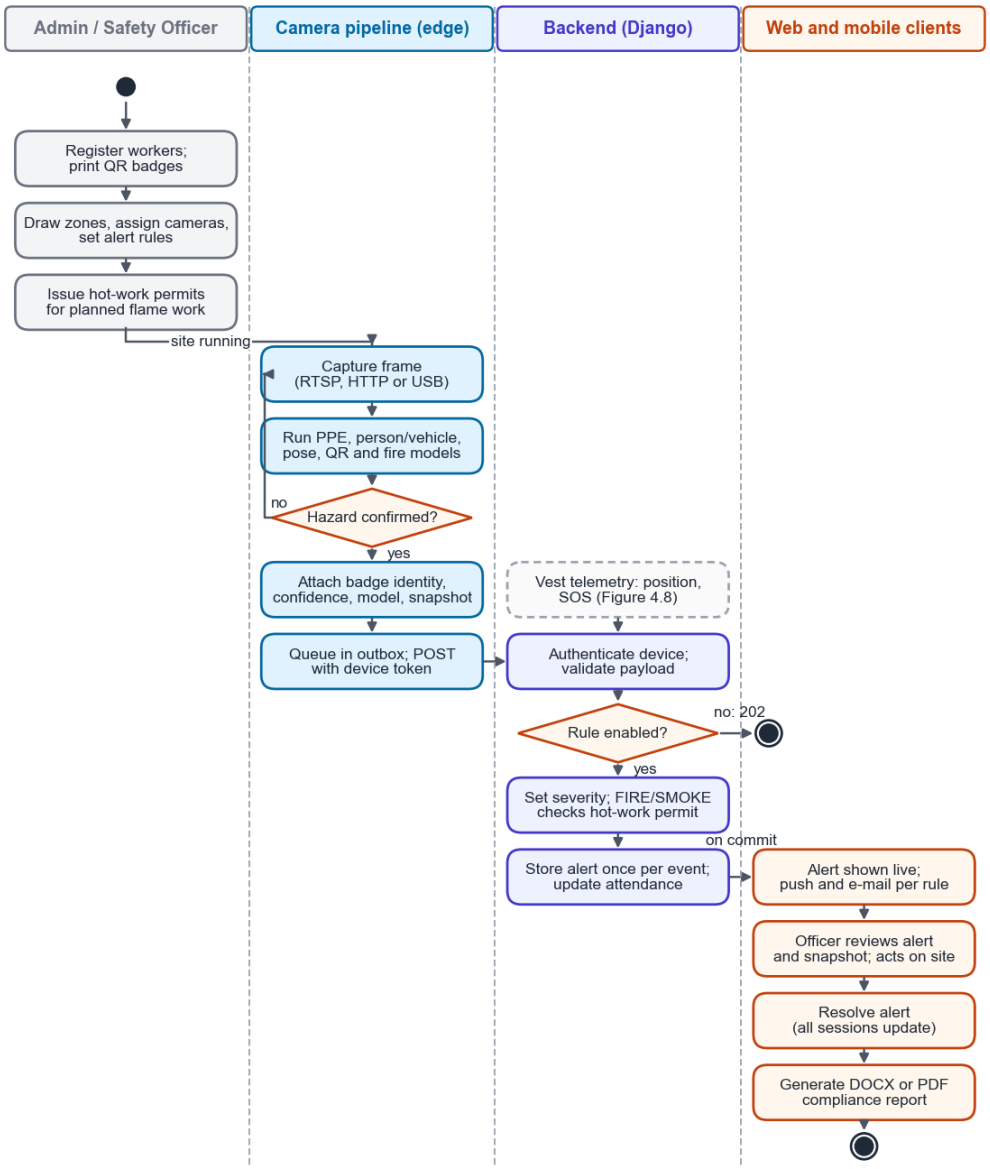


Figure 3.9: System Workflow Activity Diagram

Figure 3.10 is the Level-0 data flow diagram: the whole system as one process and the data it exchanges with its external entities. Figure 3.4 decomposes it into Level-1 processes.

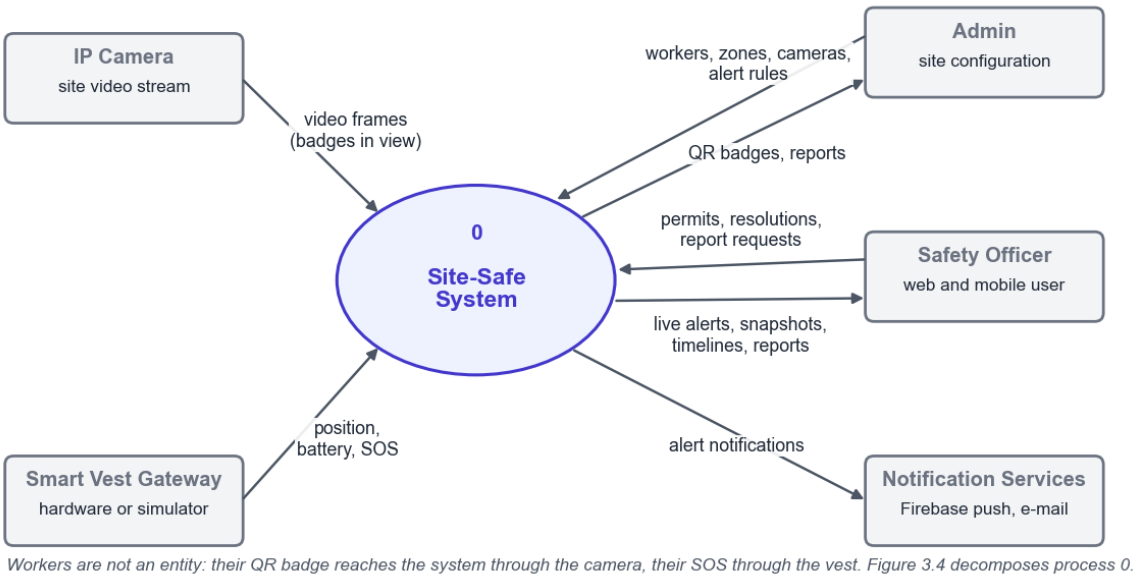


Figure 3.10: Level-0 Data Flow Diagram

3.10 Limitations

QR code identification depends on camera distance and angle: at ranges beyond approximately 10 metres, or when helmets are tilted away from the camera, the QR may not be readable. Strategic camera placement at key checkpoints mitigates this, but full-site individual tracking is not guaranteed. Continuous camera monitoring raises privacy concerns that require transparent worker consent processes and clear data handling policies. The supplementary Smart Vest provides GPS and SOS coverage outside camera zones, but relies on LoRa signal propagation which can be degraded by dense metal scaffolding. The workforce activity monitoring component can only track workers in areas with camera coverage.

Chapter 4

Hardware and Software Implementation

4.1 Development Environment

Site-Safe was developed and measured on one Apple M4 laptop with 16 GB of memory running macOS; the versions in Table 4.1 were read from the project environments and lock files on 20 October 2026. The PPE model was trained on a Google Colab Tesla T4 GPU and the fire model on a Kaggle T4, because training on the laptop ran out of memory. A docker-compose file and a GitHub Actions workflow are written but have not yet been run.

Table 4.1: Development Environment Summary

Tool / Platform	Version	Purpose
Python	3.14.3	Backend, camera pipeline and measurement tools
Django / Django REST Framework	6.0.4 / 3.17.1	REST API, permissions and serializers
Django Channels / Daphne	4.3.2 / 4.2.3	WebSocket delivery; ASGI server for REST and WebSocket
PostgreSQL	18.3	Development and measurement database; backend tests also run on SQLite
Redis	7 (container image)	Channel layer between backend processes in the compose deployment
Eclipse Mosquitto / paho-mqtt	2 (container image) / 1.6.1	Vest gateway messages read by the MQTT consumer
Ultralytics / PyTorch / OpenCV	8.4.41 / 2.11.0 / 4.13.0	Detection, pose estimation and QR decoding
Node.js / Express	25.8.1 / 5.2.1	Report service runtime
docx / pdfkit	9.7.1 / 0.20.2	DOCX and PDF report rendering
React / Vite / Tailwind CSS	19.2.8 / 8.2.2 / 4.3.3	Web command centre
Leaflet / react-leaflet	1.9.4 / 5.0.0	Site map and zone editor (OpenStreetMap tiles)
Expo SDK / React Native	58.0.12 / 0.88.0	Mobile application
react-native-webview / react-native-svg	14.0.1 / 15.16.0	Mobile site map (Leaflet) and danger-area outlines
Docker Compose / GitHub Actions	Written, not yet run	Eight-service deployment; CI running every test suite
Git	2.53.0	Version control

4.2 QR Identification and Smart Vest

4.2.1 QR Worker Identification

Each worker has a random badge code, created at registration and reissued from the Workers page, which prints a QR badge with the payload SSW1:<code> for the helmet. On the camera, identity.py decodes QR codes with OpenCV on the frames sampled for the person detector and associates each badge with the person box that contains it, following

that person between samples by box overlap. A badge inside two person boxes, or two badges inside one box, is treated as unknown rather than guessed, so people standing close together are not misattributed.

Alerts from a person with a known badge are attributed to that worker; others are marked as not identified. Sightings, repeated every 15 seconds while a badge stays in view, feed attendance (Section 4.7) and place the worker in the camera's zone (Section 4.4); unknown or revoked codes are ignored with HTTP 202 so that they cannot block a camera's queue. Reading range, angle and lighting have not yet been measured with printed badges, so the 5–10 metre range in Section 3.4.1 remains an assumption.

4.2.2 Smart Vest

The physical Smart Vest (ESP32 [13], GPS module, SOS button and LoRa link [14]) has not been built. It was moved to future work through a scope-change request to the supervisor: the vest is supplementary by design (Section 3.4.2), parts and lab access were not available in time, and the camera functions carry the primary objectives. Figure 3.6 and Tables 4.2 and 4.3 remain the proposed design.

The software path a vest would use is implemented. A gateway posts JSON telemetry (vest, position, battery and SOS state) to /api/v1/vests/telemetry/ with a gateway token, or publishes it to sitesafe/vest/<id>/telemetry and /sos on a Mosquitto broker [15] read by the MQTT consumer. Both paths validate the payload and share one handler (Figure 4.8). A vest simulator drives this path with scripted scenarios; its token is flagged, so its alerts are labelled SIMULATED, create no attendance and are excluded from reports.

Table 4.2: Proposed Smart Vest Component Specifications

Component	Model	Range	Accuracy	Interface
GPS	NEO-6M	Worldwide	2.5 m CEP	UART
SOS Button	Tactile Switch	N/A	N/A	GPIO

Table 4.3: Proposed ESP32 Pin Allocation

GPIO Pin	Connected To	Protocol	Notes
GPIO 16 (RX2)	NEO-6M TX	UART	9600 baud
GPIO 17 (TX2)	NEO-6M RX	UART	9600 baud
GPIO 18	LoRa Module SCK	SPI	SX1276
GPIO 23	LoRa Module MOSI	SPI	SX1276
GPIO 19	LoRa Module MISO	SPI	SX1276
GPIO 5	LoRa Module NSS	SPI	Chip select
GPIO 2	Status LED	Digital	Activity indicator
GPIO 15	SOS Button	Digital	Pull-down, interrupt

4.3 PPE Detection Module

The camera pipeline runs the PPE detector models/sitesafe_v5.pt, a YOLO11-medium model trained with Ultralytics [3], on every frame. A COCO-pretrained YOLO11-nano detector supplies person and vehicle boxes, and a YOLO11-nano pose model keypoints, each on every third frame; all read the same unannotated frame. Each missing-PPE box is tracked as one person's episode (ppe_episodes.py): duplicate boxes for one body part are merged, and each box is matched to its wearer through the person tracker. An episode alerts once after persisting for one second in at least three frames, and again only after 30 seconds unseen, so a flicker raises nothing and two people missing helmets raise two alerts. Every camera detector follows this pattern.

4.3.1 Dataset and Classes

The training data is the Roboflow dataset ppe-detection-qlq3d (CC BY 4.0) [4]: 5,140 images in ten classes, helmet, vest, boots, gloves and goggles, each with a matching no-class, so that a violation is detected directly rather than inferred. It has no person or vehicle class, hence the second detector.

The published split (Table 4.4) is not independent. The repository audit (audit_dataset.py) found 573 source groups and 30 groups of exact duplicates with members in more than one split, and 69 groups of identical images with conflicting labels. Test images therefore have near-copies in training, which inflates the scores below. A group-constrained re-split (3,598 / 771 / 771 images) has been generated but is held back until the label conflicts are resolved.

Table 4.4: Dataset Split Configuration

Split	Images	Percentage	Purpose
Training	3,597	70%	Model weight optimization
Validation	1,026	20%	Early stopping and model selection
Test	517	10%	Final performance evaluation

4.3.2 Training Configuration

The model was fine-tuned from COCO-pretrained YOLO11-medium weights with the settings in Table 4.5, recorded in the exported run manifest, replacing an earlier nano model trained at 416 pixels (mAP@0.5 0.544). The team's Colab record shows early stopping after 102 of 150 epochs, with the best weights at epoch 77. Figure 4.1, drawn from the exported run, summarises the path from data to deployed weights.

Table 4.5: YOLOv11 Training Hyperparameters

Parameter	Value	Rationale
Base model	YOLO11-m (medium), COCO-pretrained	Larger capacity than the earlier nano baseline
Input resolution	640 × 640	Standard Ultralytics training resolution
Batch size	16	Fits the T4's memory at 640 px
Epochs	150 budget, early stopping (patience 25)	Recorded run stopped after 102 epochs; best at epoch 77
Optimizer	AdamW	Weight decay handling
Learning rate	0.001, linear decay to 1% (lrf 0.01)	Cosine schedule disabled for this run
Augmentation	Mosaic, flip, HSV, scale (defaults)	Mosaic disabled for the final 10 epochs
Seed	42, deterministic	Repeatable run
Confidence threshold	0.15	Runtime candidate threshold; not yet calibrated per class
NMS IoU threshold	0.7	Ultralytics default

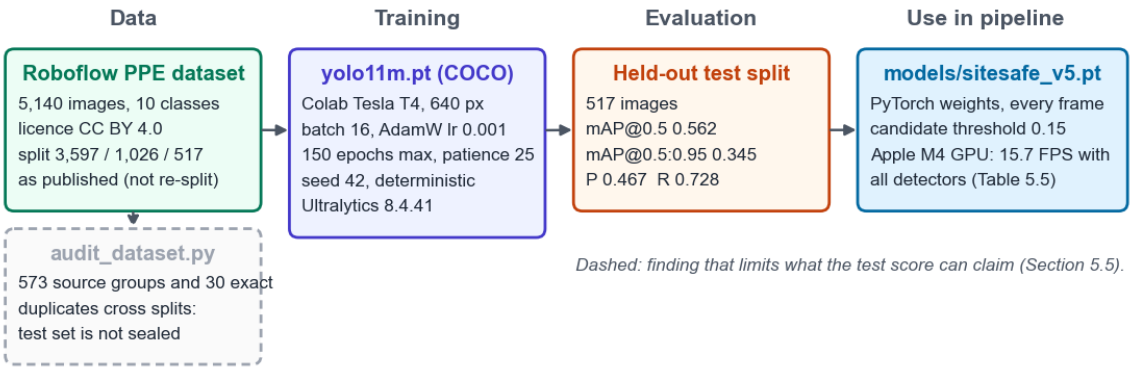


Figure 4.1: PPE Model Training and Evaluation Pipeline

4.3.3 Model Performance

On the held-out split the model reaches mAP@0.5 of 0.562 and mAP@0.5:0.95 of 0.345, with precision 0.467 and recall 0.728 (Table 4.6). Worn equipment is recognised reliably: helmet, vest and boots exceed 0.95. Missing equipment, the condition the system exists to detect, is recognised poorly (no-gloves 0.062, no-boots 0.130, no-helmet 0.309), because absence must be inferred and the negative classes are under-represented. Better data for these classes is the most valuable improvement, and the split leakage makes every figure an upper bound.

Table 4.6: Per-Class Detection Performance

Class	Precision	Recall	mAP@0.5
helmet	0.893	0.959	0.958
no-helmet	0.249	0.714	0.309
vest	0.853	0.954	0.962
no-vest	0.578	0.879	0.782
boots	0.845	0.998	0.977
no-boots	0.127	0.596	0.130
gloves	0.222	0.316	0.276
no-gloves	0.109	0.323	0.062
goggles	0.470	0.793	0.635
no-goggles	0.321	0.750	0.530
all classes	0.467	0.728	0.562

4.3.4 Fall Detection

Fall detection (fall_detection.py) runs its own pose model on every third frame, since neither the PPE nor the COCO detector gives keypoints. A posture counts as collapsed only when both signals of Section 3.3.5 agree: the confident keypoints form a box wider than tall, and the hips are at or below the knees. If either signal cannot be measured, for example with a leg out of frame, the result is not-collapsed, so missing data does not raise an alarm.

People are tracked between samples by a confidence-weighted keypoint centroid, and a fall is confirmed after three continuous seconds of collapse; it raises one FALL alert and latches until the posture clears. Twenty-four tests cover the geometry and the tracker, and standing pedestrians in real photographs were read as not collapsed. The module has not been tested on fall footage, and its pixel geometry is uncalibrated per camera.

4.3.5 Fire and Smoke Detection with Hot-Work Permits

On the camera, fire_detection.py runs on every fifth frame and confirms fire or smoke only when it persists for two seconds with gaps of at most one second, ignoring specks below 0.05 percent of the frame; after an alert it latches until the scene has been clear for 30 seconds, so a fire raises one alert, not one per frame.

Severity is set by the server (alerts/fire_policy.py) from hot-work permits, which safety officers issue for a zone and time window and can revoke. Without an active permit FIRE is CRITICAL and SMOKE HIGH; within one, FIRE is MEDIUM and SMOKE LOW, and the alert is linked to the permit. Unlike the suppression proposed in Section 3.3.6, a permitted flame is still stored and broadcast, so a spreading welding fire is not hidden, and FIRE cannot be disabled.

The fire model is a two-class YOLO11-small detector trained on the D-Fire dataset on a Kaggle T4 GPU (early stop after 96 epochs, best at 76). On 4,294 held-out images it reached precision 0.775, recall 0.720 and mAP@0.5 of 0.787 (fire 0.726, smoke 0.848). The permit policy was verified end to end: a fire in a permitted zone became MEDIUM and linked, elsewhere CRITICAL. Video and hard negatives such as welding remain to be evaluated.

4.3.6 Camera Sources and Reliable Event Delivery

The camera source is a device index or a stream URL, so an RTSP camera, an NVR or a phone running an IP-camera application uses the same code path as a webcam, and up to 30 consecutive failed reads are tolerated before a stream counts as disconnected. Each

pipeline has a camera identifier, and zones list their cameras, so each alert is attributed to a zone.

Every event is first written to a local SQLite outbox under a unique event identifier and delivered in the background with back-off capped at 30 seconds, also after a restart. The server stores each identifier once (201 new, 200 duplicate, 202 type disabled) and dead-letters invalid events. Cameras use device tokens scoped to their identifier and read their rules and danger areas every 15 seconds, so dashboard changes apply without a restart.

Each pipeline also sends its annotated view, up to five frames a second from a background thread that drops stale frames, for the live camera grid (FR-30). The API relays frames to Safety Officers' and Admins' sockets and keeps only each camera's latest frame for 30 seconds; live video is never recorded.

4.3.7 Vehicle Proximity Detection

Proximity uses the COCO detector's person and vehicle boxes. Without depth, the gap between a person box and a vehicle box is divided by the person's box height, making the threshold roughly scale-invariant (one person-height is about 1.7 metres at similar distances). Each tracked person has one encounter, confirmed after three samples near a vehicle with no gap over one second; it raises one alert, CRITICAL if the boxes overlap and HIGH otherwise, and ends after 15 seconds away from every vehicle.

The first design re-armed whenever one sample missed the vehicle, limited only by a camera-wide 15-second cooldown: one worker beside a flickering car raised 14 alerts in three minutes, and two workers at risk shared one. Replaying the same detections, encounters cut a three-loop clip from 10 alerts to 2, one per person (Table IV.3). COCO has no forklift class, so plant is reported as a truck or car, and a worker in front of a distant vehicle is over-reported.

4.3.8 Danger Areas

Hazards smaller than a zone, such as a crane's swing radius, are drawn by Admins as polygons on a camera's live picture and stored as fractions of it (danger_areas.py). A person is inside when the bottom-centre of their box, their feet, lies in the polygon, so someone standing in front of an area is not flagged. One DANGER_AREA alert is raised per person per entry after one second inside, and again only after ten seconds outside; the camera fills

an occupied area red. Areas can be switched off while the hazard is absent, from the web or the phone.

4.4 Backend Implementation

The backend is a Django 6 and Django REST Framework [10] application on PostgreSQL 18, served by Daphne so that REST and WebSocket traffic share one ASGI process through Django Channels [22]. Its five applications are accounts, alerts, workers, sitemap (zones and permits) and reports. Table III.1 lists the endpoints, Table III.2 the schema and Figure 4.2 the data model, drawn from the Django models.

Users receive a 30-minute JWT access token and a refresh token, renewed silently, and Admin, Safety Officer and Viewer permissions are enforced on every endpoint. Producers use device tokens instead: only a hash of each key is stored, each has a kind and permitted cameras, and none can sign in, so no write endpoint is anonymous. The WebSocket requires a valid user token and an allowed origin and otherwise closes with code 4401, telling clients to renew and reconnect.

Changes are broadcast after the transaction commits, so clients never see a rolled-back alert, through a Redis channel layer between processes. Alert rules set, per type and optionally per zone, whether an alert is enabled, its threshold and whether it notifies; SOS and FIRE cannot be disabled. Notifications go out after commit through Firebase Cloud Messaging [12] and e-mail. A presence monitor marks vests offline after 120 seconds of silence and closes attendance sessions.

Cameras also locate workers who wear no vest (workers/location.py): a badge sighting places the worker in that camera's zone, current for 60 seconds and then shown as last seen. Each camera reports how many people it sees, and a zone's head count is the larger of its identified workers and its cameras' counts, combined by the largest count when their views overlap, so nobody is counted twice. Over-capacity alerts use hysteresis, ten seconds over the limit to alert and sixty within it to re-arm, because head counts are noisy.

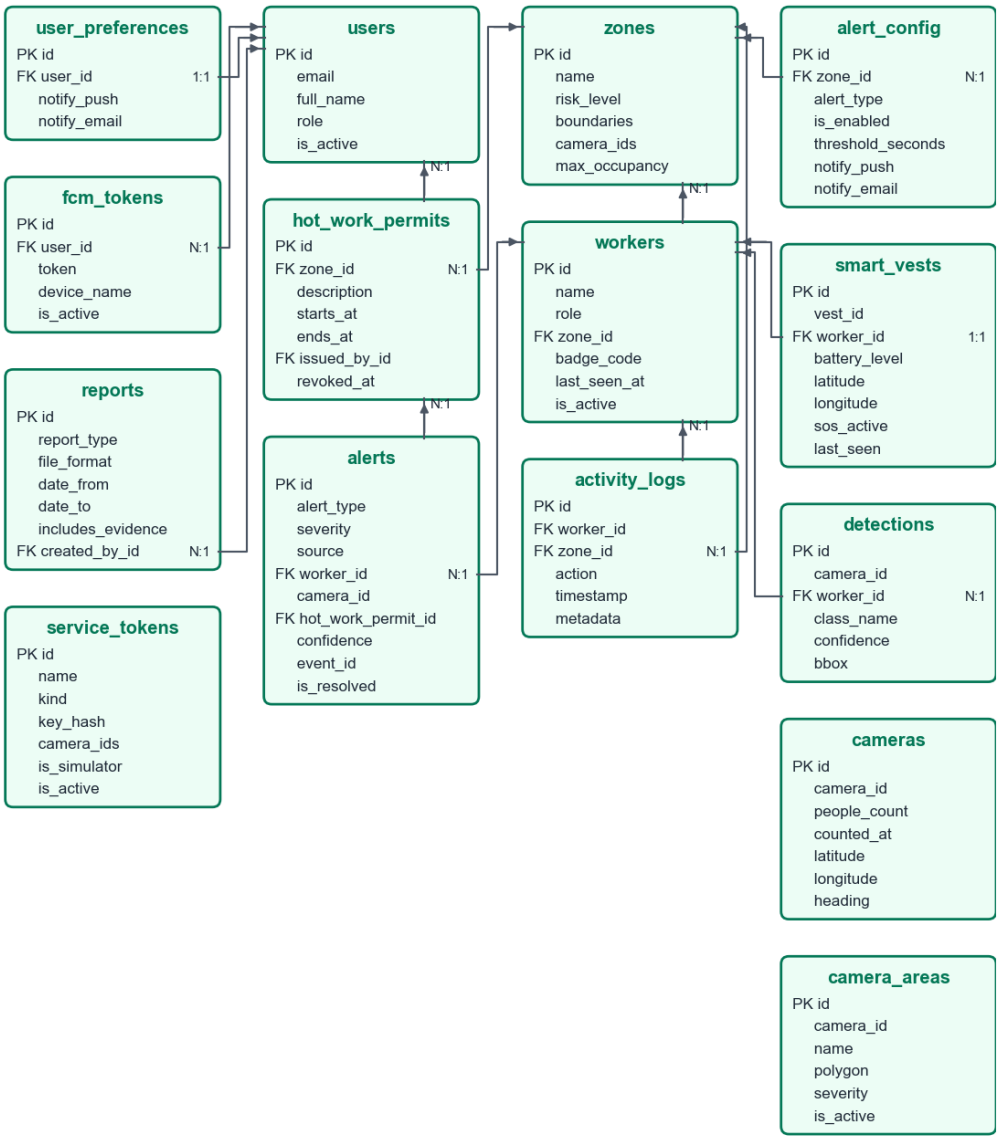


Figure 4.2: Entity Relationship Diagram

4.5 Frontend Dashboard Implementation

The web command centre is a React 19 application built with Vite and Tailwind CSS, with routes for the dashboard, workers, incidents, vests, zones, reports and settings. Every dashboard figure comes from stored records; the seeded charts, vitals and compliance percentage of the earlier prototype were removed. A WebSocket adds and removes alerts live, and the client resynchronises after a reconnect.

The Cameras page shows every camera's annotated live view to Safety Officers and Admins; Admins draw danger areas on its enlarged view (Figure 4.3, left). On the Zones page's Leaflet [23] map, Admins draw zone polygons with risk level, occupancy limit and

cameras, issue hot-work permits, and place each camera with its direction, view angle and range. The dashboard repeats the site map with live head counts, vests and cameras, which turn red for a minute after an alert (Figure 4.3, right). The Workers page registers workers, assigns vests, prints badges and shows daily timelines.

Reports are generated in DOCX or PDF for a period in the user's time zone and give counts, resolution times, violation-free days and attendance, each defined, but no compliance percentage, because the system records violations and not compliant observations, so no denominator exists. They can include the evidence frames of the most severe incidents, captioned, numbered and referenced from the incident log, and state how many incidents had no frame.

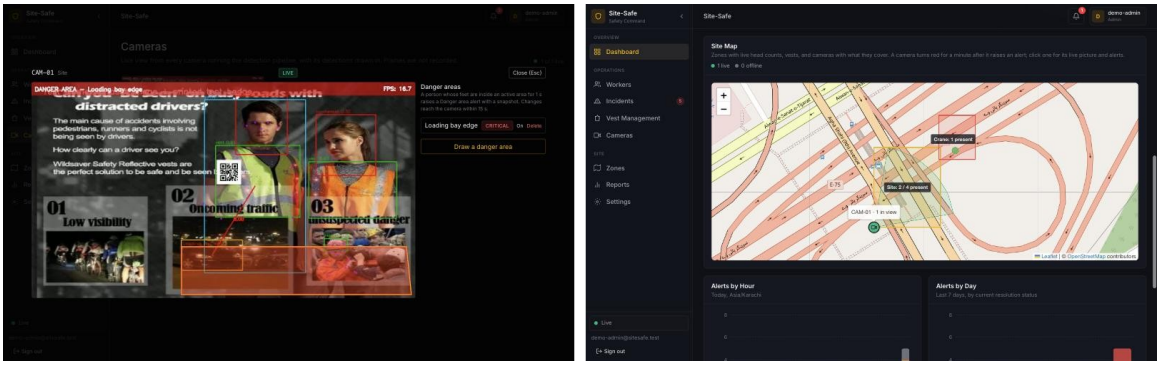


Figure 4.3: Web Command Centre: Live Camera with a Danger Area, and the Site Map with Cameras

4.6 Mobile Application

The mobile application uses React Native 0.88 [11] on Expo SDK 58 in TypeScript (Section 3.6). Expo builds Android and iOS clients without Xcode or Android Studio and supplies the FCM device token; the app uses the same REST API and WebSocket as the web dashboard.

4.6.1 Technology Stack

Table 4.7: Mobile Application Technology Stack

Component	Technology	Purpose
Framework	React Native 0.88 on Expo SDK 58 [21], TypeScript	Android and iOS from one codebase
Navigation	Expo Router (file-based tabs and stack)	Alerts, Map, Cameras, Workers and Settings tabs; alert and camera screens
Map and overlays	Leaflet 1.9.4 in react-native-webview; react-native-svg	Same site map as the web; danger areas over the live picture
State	React context and reducers	Live alert and worker state
HTTP client	fetch wrapper with single-flight token refresh	REST calls; sign-out when renewal fails
Live updates	WebSocket with 4401 renewal, back-off and resync	Same live stream as the web dashboard
Push notifications	expo-notifications 58.1 (FCM on Android)	Alert delivery; tap opens the alert
Secure storage	expo-secure-store 58.0	Tokens in the device keychain/keystore
Quality checks	jest-expo (25 tests), ESLint, TypeScript	Run in CI

4.6.2 Core Screens

After sign-in, five tabs follow (Figure 4.4). Alerts updates live; an alert shows its evidence frame, confidence and model version, links to its camera, and can be resolved by Safety Officers and Admins. Map shows the web's Leaflet site map in a WebView, so Android needs no map key. Cameras lists each camera's status and people in view; a camera's screen shows its live picture with danger areas outlined, and Admins can switch an area off. Workers shows presence and location, and Settings controls push. Reports, zone editing and worker registration stay on the web; the activity summary, rule configuration and shift scheduling of Figure 3.8 were not built.

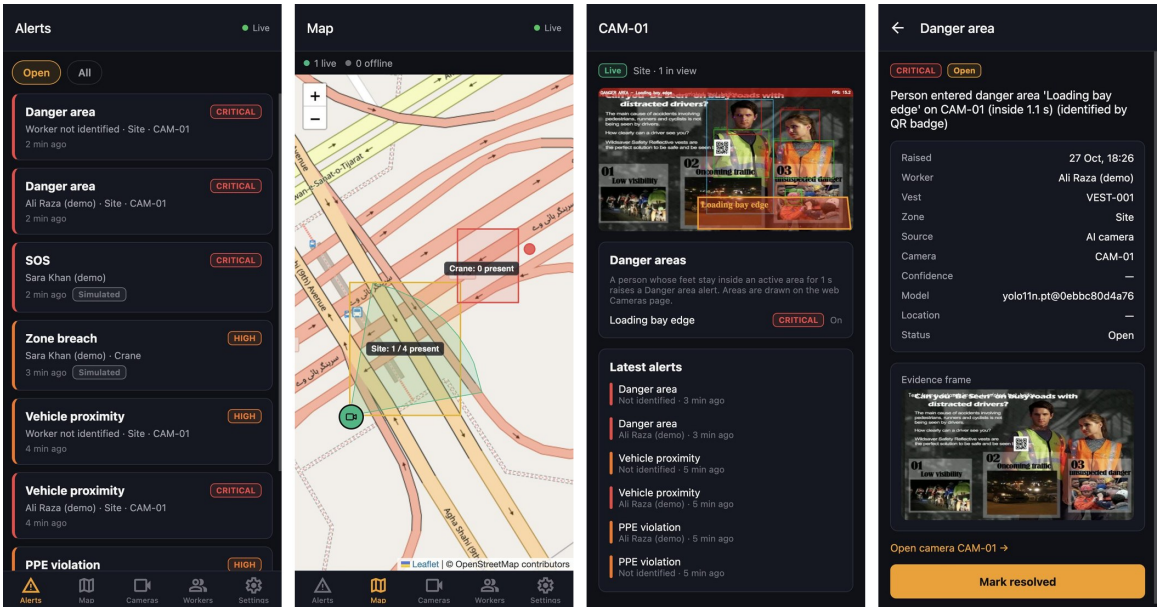


Figure 4.4: Mobile Application: Alerts, Site Map, Camera and Alert Detail

4.6.3 Push Notification Flow

On Android the app registers its FCM token after sign-in and removes it on sign-out, and tapping a notification opens the alert. Both sides are tested with a mocked FCM service, but delivery to a real phone needs a Firebase configuration and a development build and has not been verified; iOS relies on the live socket. Figure 4.4 comes from the app's web build, on which every screen was exercised against a test camera; the native map and overlay modules have not yet run on a phone.

4.7 Workforce Activity Monitoring

Inactivity tracking (inactivity.py) follows people between samples by centroid; a track that stays within a movement threshold for the timeout set in the dashboard rules raises one INACTIVITY alert and re-arms only after moving, and brief occlusions do not drop it. Displacement is in uncalibrated pixels.

Identified workers get attendance automatically (workers/activity.py). The first evidence of a worker, a badge sighting or a vest report, checks them in; idle start and end come from the inactivity tracker; and after 30 minutes without evidence the presence monitor checks them out at the time they were last seen. Late events, such as those replayed from an outbox, are merged into their session. A timeline API gives per-worker daily intervals of presence and idleness, shown on the Workers page and in the report attendance tables.

Twenty-three tests cover check-in, check-out, idleness and late events; an end-to-end run gave 60 minutes on site and 20 idle, identical in the timeline and the PDF. Records exist only for workers with a readable badge or a vest, and identities can swap when identified people cross.

4.8 System Architecture and Deployment

Figure 4.5 shows the implemented components. The camera pipeline and the vest gateway or simulator write through authenticated HTTP or MQTT, the API is the only writer to PostgreSQL, and clients read through REST and receive changes over the WebSocket. The three tiers of Figure 3.3 remain, but notifications use an after-commit thread pool instead of Celery, documents come from a Node.js report service, and the edge runs on a laptop GPU instead of a Raspberry Pi or Jetson.

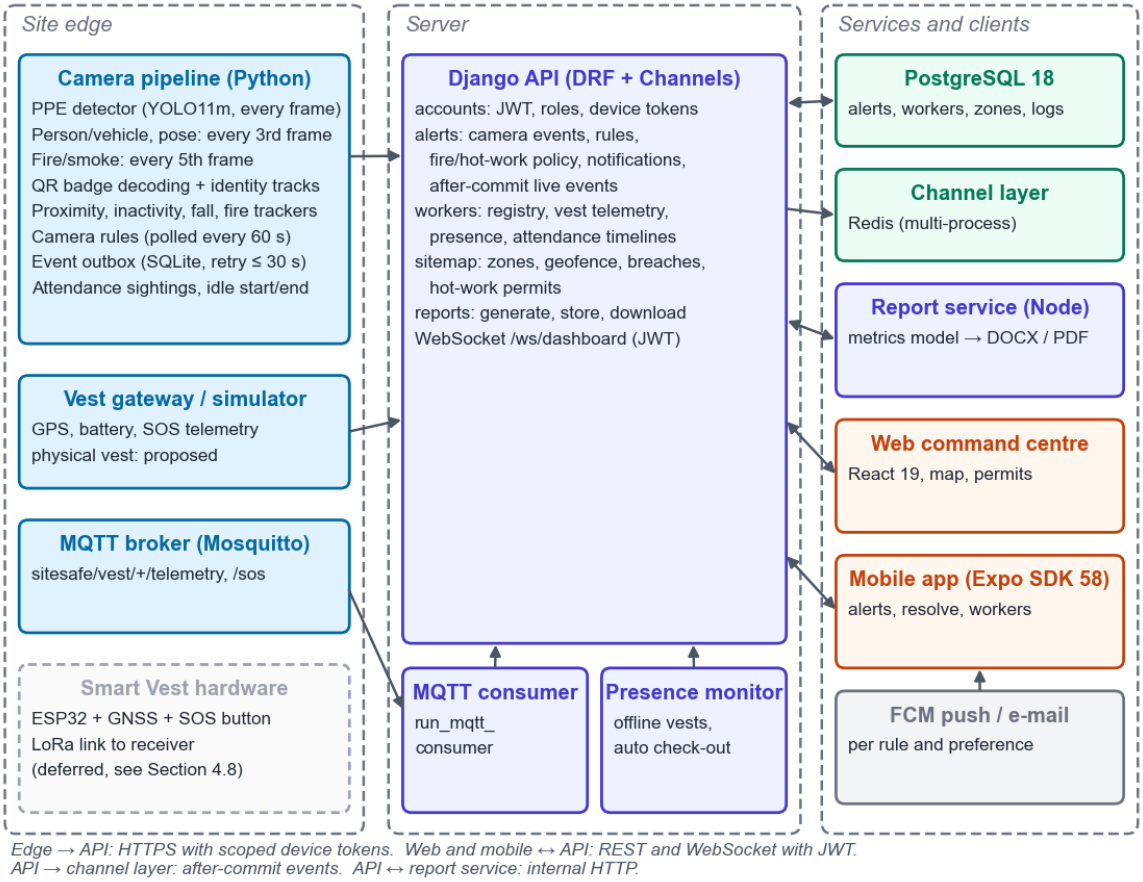


Figure 4.5: Component View of the Implemented System

Figure 4.6 shows the demonstration deployment: the camera pipeline runs natively to use the GPU, and the server is defined as eight docker-compose services. The compose file has not yet been run, so Chapter 5's results come from the same services started on the host.

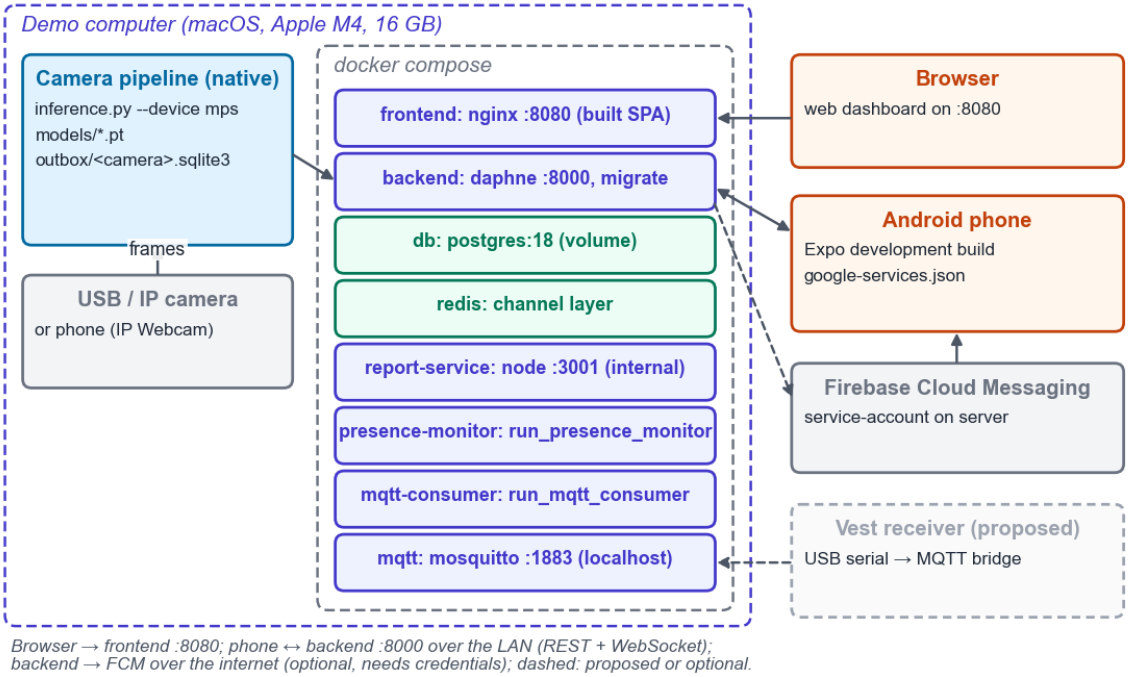


Figure 4.6: Deployment View

Figure 4.7 shows the camera pipeline's structure: main() owns the capture loop and one helper per concern, each free of camera and model imports so that it can be unit-tested without OpenCV, a GPU or weights. The script that draws Figures 4.2 and 4.7 checks every name against the code, so the figures cannot drift from the implementation.

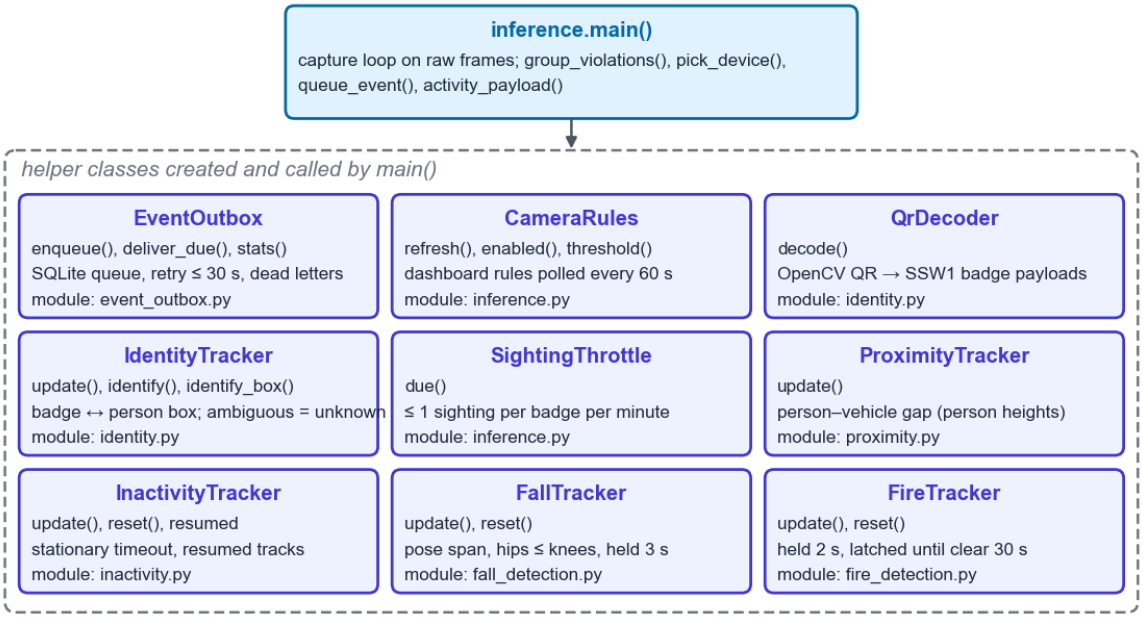


Figure 4.7: Camera Pipeline Class Structure

4.9 SOS and Location Telemetry Flow

Figure 4.8 replaces the proposed flow of Figure 3.7 with the implemented sequence. After validation, the handler locks the vest row, updates its position, checks the worker in if needed, finds the highest-risk zone at the position, logs zone entry and raises ZONE_BREACH for an unassigned entry into a restricted zone or for over-occupancy. SOS raises a CRITICAL alert on its rising edge only, so a held button does not repeat it. Changes are broadcast after commit, and the presence monitor later marks a silent vest offline.

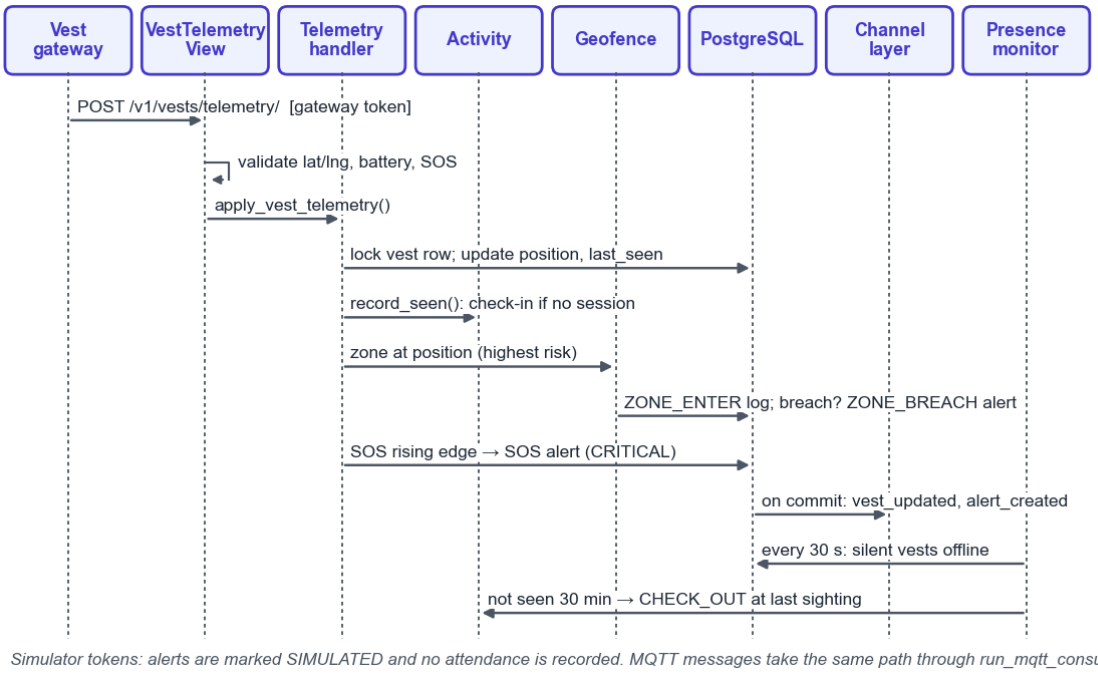


Figure 4.8: Vest Telemetry and SOS Sequence

4.10 End-to-End Alert Flow

Figure 4.9 traces a camera alert. The tracker confirms the event, the camera queues and posts it, and the server applies the alert rule and, for fire and smoke, the permit policy before storing it under its event identifier. After commit, the alert is broadcast to web and mobile clients and notifications are sent. The camera deletes the event only after the server replies, so a network failure leaves it queued. Section 5.6 measures this path.

Each AI alert carries its evidence (FR-29): the confidence, the weights that produced it (file name and the first 12 hex digits of its SHA-256) and a JPEG of the unannotated frame that confirmed it, at most 640 pixels wide, sent with the event from the outbox. The server accepts a JPEG of up to 512 KB, stores it once per event and serves it only to signed-in users; snapshots can be switched off, and a daily command deletes them after a retention period (30 days by default) while keeping the alert.

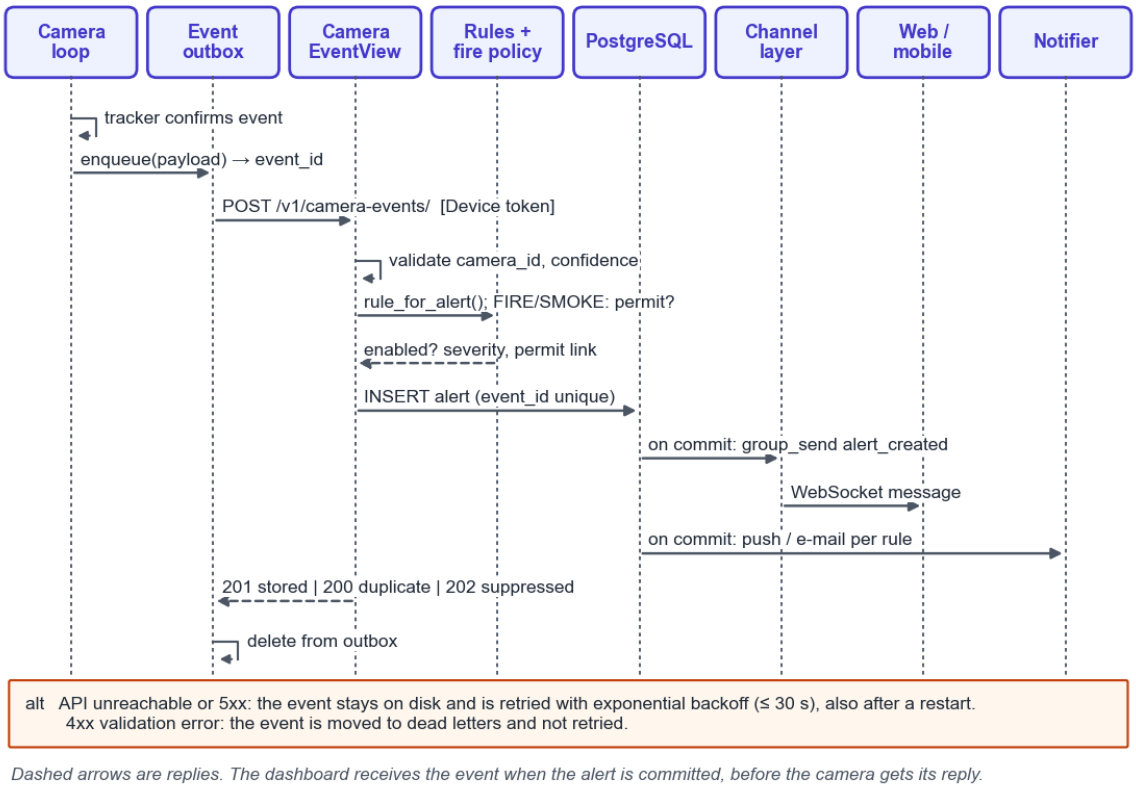


Figure 4.9: Camera Event to Dashboard Sequence

4.11 Project Timeline

The final semester followed the completion plan: securing producers and live events, replacing fabricated dashboard data, worker, zone and breach features, QR identification, reports, the mobile application and the vest simulator, then PostgreSQL verification, CI, fire and smoke with permits, attendance and the Chapter 5 measurements. The final weeks merged the codebases, made PPE and proximity alerts per person after live runs, and added the camera features of Sections 4.3.6–4.3.8, cameras on the site map and report evidence, each verified with a test video and recorded in the traceability matrix.

Chapter 5

Testing, Results and Discussion

5.1 Testing Approach

Testing was organised around the requirements traceability matrix in the repository, which links each functional (FR) and non-functional (NFR) requirement to its code, tests and evidence. A requirement's status rises through five levels: Coded, Tested (automated tests cover it), Integrated (verified end to end across running services), Physical (verified with real hardware or a real camera scene) and Accepted (agreed with the supervisor). No requirement has reached Physical or Accepted yet, and the acceptance targets are the team's proposals.

Four kinds of evidence are used. Automated unit and integration tests run on every change. End-to-end exercises start the real services and drive them with real payloads: camera events, vest telemetry, browser sessions and app sessions. Measurement scripts in tools/measure/ produce the performance and reliability tables; each saves its raw results together with the machine, the git commit and its settings, and a table generator turns them into the tables below, so no number is copied by hand. Model evaluation uses Ultralytics on the held-out test split.

5.2 Test Environment

All tests and measurements ran on the development machine: an Apple M4 laptop with 16 GB of memory running macOS, with Python 3.14.3, Node.js 25.8.1 and PostgreSQL 18.3. The backend suite runs on both SQLite and PostgreSQL, because several defects appeared only on PostgreSQL (Table IV.3). Measurements used simulator device tokens, so their alerts are labelled SIMULATED and never reach reports, and no measurement yet crosses a network to a second device.

5.3 Automated Test Results

Table 5.1 lists the automated suites. All 470 tests passed when they were re-run for this report on 27 October 2026, up from 59 in the previous revision. Decision logic on the camera side (PPE episodes, fall, inactivity, proximity and fire confirmation) is kept free of OpenCV and model imports, which lets it run under the backend test command without a camera, a GPU or weights. The web frontend has no unit tests; its lint check and production build are run instead, and its behaviour was checked in the browser (Table IV.2).

Table 5.1: Automated Test Suites

Suite	Tests	What it covers	Result
Backend (Django)	241	Roles and permissions; camera and vest ingest, device tokens and alert rules; alert evidence and retention; live camera view and camera map; camera location, head counts and capacity; danger areas; WebSocket authentication and broadcasts; workers, vests, badges, presence and breaches; activity and timelines; zones and permits; reports; demo site set-up	Pass on SQLite and PostgreSQL 18
Camera pipeline	144	PPE episodes and person association, QR identity, event outbox, alert evidence, fire confirmation, proximity encounters, danger areas, live view and people counts, inference configuration, activity events, dataset tools	Pass
Fire dataset preparation	7	Label remapping, video-grouped split, negatives, parquet unpacking	Pass
Report service (Node)	24	Metrics, evidence selection and image loading, DOCX and PDF rendering, attendance tables, pagination	Pass
Mobile app	25	API client and token refresh, push registration, live-state reducers, camera status, view cones, site-map data and messages	Pass; lint and type check clean
Tools	29	Vest simulator, measurement statistics, tables and clip scoring	Pass
Web frontend	—	No unit tests; ESLint and production build	Lint clean; build OK
Total	470		All pass (27 Oct 2026)

5.4 Requirement-Linked Test Cases

Table IV.1 condenses the traceability matrix: each requirement, its evidence and the status reached, not rounded up. FR-04 (PPE detection) remains Coded although the model runs, because its criterion, event precision of at least 90 percent and episode recall of at least 85 percent, needs a sealed set from the target camera, which does not exist yet. FR-05 is Tested: live webcam runs found one person alerted repeatedly, because one body part came as overlapping boxes and a head moving towards the camera outran its episode; merging boxes and matching them to the tracked person fixed both on the recorded runs, but no annotated video has been scored. FR-30 to FR-34, added last, are Integrated on test videos, not yet on a site camera.

5.5 Model Evaluation Results

Table 4.6 gives the held-out results of the deployed YOLO11-medium weights: mAP@0.5 of 0.562, mAP@0.5:0.95 of 0.345, precision 0.467 and recall 0.728. Present-equipment classes exceed 0.95 mAP@0.5, while no-gloves (0.062), no-boots (0.130), gloves (0.276) and no-helmet (0.309) are weak. Because a missing item is the alert condition, the aggregate overstates usefulness for the core task unless the negative classes are read separately. The

fire and smoke model (Section 4.3.5) reached mAP@0.5 of 0.787 on 4,294 held-out images, but it has not been evaluated on video or against hard negatives such as welding.

These figures are an upper bound for two reasons. First, the published split is not independent (Section 4.3.1): 573 source groups and 30 exact-duplicate groups span the training and test splits, so part of the test score measures recall of near-copies. Second, image-level mAP is not alert quality. An alert also depends on the candidate threshold, the grouping of detections per person, persistence and cooldown, none of which has been evaluated on video. A defensible accuracy claim needs the group-constrained re-split and, above all, a sealed set of footage from the target camera, including hard negatives, scored per event.

5.6 Performance and Reliability Measurements

Tables 5.2 to 5.5 report the non-functional measurements. Each table states its sample size and settings, and every run is kept, including failures, so a result can be traced to the commit and machine that produced it.

Latency (NFR-01): 100 camera events were posted every 0.2 seconds with a simulator token, and the time from sending each event to its arrival on an authenticated dashboard socket was recorded. All 100 arrived, with a 95th percentile of 41.6 ms against a proposed target of 2 seconds (Table 5.2). The dashboard can receive an event slightly before the camera receives its HTTP response, because the server broadcasts when the alert commits. The figure is for a single host; latency to a phone or a second computer on a LAN has not been measured.

Table 5.2: Event-to-Dashboard Latency (NFR-01)

Measure	n	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)
API response	100	34.2	44.0	82.2	82.3
Event to dashboard	100	32.0	41.6	80.0	81.1

Outage recovery (NFR-02): 100 events were sent while the API was unreachable for five minutes, simulated by switching off a proxy, and the camera's sender process was restarted midway. The target is that every event is stored exactly once within 60 seconds of the API returning. The first run stored all 100 exactly once but took 60.3 seconds, a failure caused by the 60-second retry ceiling; the ceiling was lowered to 30 seconds and two further runs recovered in 30.0 and 30.2 seconds with nothing lost or duplicated (Table 5.3).

Table 5.3: Outage Recovery, 100 Events over a 5-Minute Outage (NFR-02)

Run (UTC, 20 Oct)	Retry ceiling (s)	Queued at restore	Recovery (s)	Stored once	Lost	Duplicated	Result
11:51	60	100	60.3	100	0	0	Fail
12:01	30	100	30.0	100	0	0	Pass
12:13	30	100	30.2	100	0	0	Pass

Throughput (NFR-04): the full detector set was timed on the 517 test images at 1280×720, detection and decoding only, so camera frame rates are at most these figures. On the laptop GPU the pipeline reached 13.4 and 15.7 FPS against a target of 10, and 15.8 and 16.1 FPS once the trained fire model was added on 27 October; on the CPU it managed about 10 FPS, so the GPU became the default (Table 5.4). On the GPU the PPE model takes a median of 44.8 ms per frame, and the person/vehicle, pose, QR and fire stages 15–18 ms on the frames they run.

Table 5.4: Inference Throughput with All Detectors (NFR-04)

Device	Run (UTC)	Fire model	Frames	FPS	Frame p50 (ms)	Frame p95 (ms)	Result (≥ 10 FPS)
CPU	20 Oct 11:52	No	150	9.8	80.4	145.5	Fail
CPU	20 Oct 12:15	No	150	10.1	82.1	147.2	Pass (marginal)
CPU	27 Oct 09:31	Yes	150	10.2	75.9	157.6	Pass (marginal)
GPU (mps)	20 Oct 11:51	No	300	13.4	52.1	116.5	Pass
GPU (mps)	20 Oct 12:14	No	300	15.7	46.4	106.1	Pass
GPU (mps)	27 Oct 09:30	Yes	300	15.8	54.4	109.1	Pass
GPU (mps)	27 Oct 09:31	Yes	300	16.1	54.0	105.9	Pass

Soak (NFR-05): the stack ran for two hours on 22 October with the API, a camera pipeline and the presence monitor, a probe every five seconds and a forced disconnect (Table 5.5). None of 1,431 probes was missed, the 95th percentile stayed at 39.7 ms, there were no unexpected socket drops, and the client reconnected 1.1 seconds after the forced disconnect. API memory ended lower than it started (85 to 67 MB, peak 141 MB), so the rise seen in the five-minute validation run was warm-up, and NFR-05 is met on a single host.

Table 5.5: Soak Validation Run (NFR-05)

Duration	Probes	Missed	p50 (ms)	p95 (ms)	Socket drops	Forced reconnect (s)	API memory (MB)
5 min (validation, 20 Oct)	146	0	35.9	41.1	0	1.2	63.6 → 91.6 (max 109.8)
2 h (22 Oct)	1,431	0	24.4	39.7	0	1.1	85.2 → 67.2 (max 141.1)

5.7 End-to-End Verification

Table IV.2 records the end-to-end exercises, each against real running services. They show that the parts exchange real payloads correctly, but each has a stated boundary: the camera features used test videos rather than a site camera, and none used a phone receiving push or an assembled vest.

5.8 Defects Found by Testing

Testing and measurement found the defects in Table IV.3 before they reached a demonstration. Three of them appeared only when the backend ran on PostgreSQL instead of the SQLite test database, which is why both now run, and two came from the measurements, which is why each measurement keeps its failed runs.

5.9 Discussion and Threats to Validity

The software results are strong within their scope. Producers can no longer write anonymously, events survive outages and restarts exactly once, live updates reach clients in tens of milliseconds, and the attendance and report figures reconcile with the stored records. The detection results are weaker. The PPE model is good at confirming that equipment is worn and poor at noticing that it is missing, and the held-out score is inflated by the dataset split, so the system's core detection claim is the least proven part of the project.

Several threats limit how far the results generalise. All measurements ran on one machine, so network effects are absent. Camera behaviour was exercised with test videos, a webcam, still images and injected events, not footage of a real site; QR range and real falls have not been measured, and fire detection only on images. The vest was simulated, push to a phone and the containerised deployment are untested, and the team set the targets. The evidence supports an integrated prototype whose software reliability is measured and whose detection accuracy must still be established in the field, not a production-ready system.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project set out to replace intermittent manual safety oversight on construction sites with a continuous, camera-first system that identifies the workers involved and keeps an auditable record, and the prototype does this end to end in software. Cameras run PPE, person and vehicle, pose, fire and QR models, watch danger areas drawn on their picture, place identified workers in zones, and post events with their evidence through an outbox that survives outages. A Django backend applies alert rules and hot-work permits, keeps attendance, notifies supervisors and produces DOCX and PDF reports, and web and mobile clients show the results live, with every camera on the site map.

The measurements support the reliability of that software on a single host: a 95th-percentile event-to-dashboard latency of 42 ms, exactly-once delivery of 100 events across a five-minute outage with recovery in about 30 seconds, 16 frames per second for the full detector set on the laptop GPU, a two-hour soak without a missed probe, and 470 passing automated tests. They do not yet support a claim about detection accuracy on a real site. The PPE model recognises missing equipment poorly, its test score is inflated by a leaking dataset split, and fall, fire and QR detection have not been evaluated on real footage. Table 6.1 compares each objective with what was delivered.

6.2 Objectives and Outcomes

All ten objectives have working software. Eight are integrated or tested end to end, one (the PPE model) is evaluated offline only, and the physical Smart Vest of objective 8 was moved to future work. The remaining gaps are evaluation with real cameras and devices rather than missing features.

Table 6.1: Objectives and Outcomes

Objective (Section 1.3)	Delivered	Evidence reached	Outstanding
1. PPE detection model	YOLO11m, 10 classes, every frame; one alert per person per episode	Offline evaluation, upper bound (mAP@0.5 0.562); live webcam runs	Sealed target-camera set; data for negative classes
2. QR worker identification	Badges, decoding, attribution, zone placement	Integrated on test videos	Physical range, angle and lighting test
3. Geofencing	Map polygons; breach and capacity from GPS and cameras; danger areas on camera pictures	Integrated with test videos and simulated GPS	Real GPS; site cameras
4. Web command centre	Live dashboard, camera grid, site map with cameras, permits, reports with evidence	Integrated in the browser	Use on a site network

Objective (Section 1.3)	Delivered	Evidence reached	Outstanding
5. Mobile application	Alerts, resolve, workers, site map, cameras, push registration	Integrated on the web build	Android build and push delivery; rule configuration
6. Workforce activity	Inactivity, automatic check-in/out, idle, timelines	Integrated with synthetic events	Annotated real-camera scenario
7. Incident logging and reports	DOCX and PDF with defined metrics and attendance	Integrated	Supervisor sign-off on the metrics
8. Smart Vest	Telemetry API, MQTT consumer, simulator	Software integrated; hardware moved to future work	Build and field-test the vest
9. Fall detection	Pose geometry with 3 s persistence	Tested	Evaluation on fall video
10. Fire and smoke detection	Trained model, confirmation logic and permit policy	Integrated; image-level evaluation (mAP@0.5 0.787)	Evaluate on video with hard negatives

6.3 Limitations

The main limitations follow from the evidence. Detection quality is bounded by a public dataset that does not represent the target cameras, and its split leaks. Proximity and inactivity are measured in pixels on uncalibrated cameras, danger areas assume people stand on the ground the area was drawn on, and a camera's cone on the map is entered by hand. Worker identity depends on a readable badge and can swap when identified people cross. A crash between storing an alert and sending its notification loses the notification, though not the alert. The system serves one site, the map needs internet access for its tiles, iOS receives no remote push, and all measurements come from one machine.

6.4 Future Work

The first priority is the data behind detection. The 69 conflicting label groups should be resolved so the group-constrained split can be used, footage should be collected from the target cameras with hard negatives and the weakest classes, per-class thresholds calibrated, and alerts scored per event and episode on a sealed set, which would also measure the episode tracking of FR-05 on video. The fire and smoke model should be evaluated on recorded clips, including welding and other hard negatives.

The second priority is physical verification: a QR range and lighting test with printed badges, an Android development build receiving push notifications and showing the map and camera screens, latency to a second device over a LAN, a staged fall recording, and a run of the docker-compose deployment and the CI workflow. The measurement scripts already exist, so each of these is a matter of running them on the demonstration setup.

Beyond the FYP, the Smart Vest can be built against the existing gateway interface: an ESP32 with GPS and an SOS button, a point-to-point LoRa link to a USB receiver that bridges to MQTT, and field tests of range, delivery delay and battery life. Other extensions are a durable queue for notifications, alert-rule configuration in the mobile app, and support for more than one site.

Appendix I

User Manual

This appendix explains how each role uses Site-Safe once it is installed (Appendix II). The web command centre is used for set-up and monitoring (Table I.1) and the mobile app for responding on site (Table I.2). Controls appear only for the roles that may use them. The full manual, with screenshots, is docs/USER_MANUAL.md in the repository.

Table I.1: Web Command Centre Tasks

Task	Role	Steps
Sign in	All	Open the dashboard address; enter e-mail and password. The session renews itself while in use.
Monitor the site	All	Dashboard: counts and charts from recorded data, live alerts, worker status and the site map with every camera.
Watch cameras	Safety Officer, Admin	Cameras: live view of every camera with its detections; click a camera to enlarge it.
Draw a danger area	Admin	Cameras → open a camera → Draw a danger area: click the corners where people stand, name it, choose a severity, Save area. On/Off and Delete beside each area.
Register a worker and print a badge	Safety Officer, Admin	Workers → Add worker: full name, trade, phone, assigned zone and smart vest. Edit the worker → Print badge; print at 100% (60 mm) and fix it to the helmet front. Reissue badge revokes earlier prints.
Follow a worker's day	All	Workers → open a worker → timeline of time on site and idle time for a chosen day.
Manage vests	Safety Officer, Admin	Vest Management: register a vest, assign it to a worker, see battery and online state.
Draw a zone	Admin	Zones: draw the polygon on the map, then set name, risk level, maximum occupancy, cameras and whether their views overlap.
Place a camera on the map	Admin	Zones → Cameras → Place: click where it is mounted; set facing, view angle and range; Save position.
Issue a hot-work permit	Safety Officer, Admin	Zones → select the zone → Hot-work permits: description, start and end (up to 12 hours). Revoke ends it early.
Review and resolve incidents	Safety Officer, Admin	Incidents: filter by type, status and date; open an alert to see snapshot, confidence and model version; Resolve. Export CSV saves the list.
Generate a report	Safety Officer, Admin	Reports: choose incident or safety-performance report, period and DOCX or PDF; Include evidence snapshots adds the frames of the most severe incidents; Generate, then download.
Configure alert rules	Admin	Settings → Alert Configuration → edit a rule: enabled, threshold in seconds, push and e-mail. SOS and FIRE stay enabled.
Notifications and password	All	Settings → Notifications to opt out of push or e-mail; Settings → Profile to change the password.

Table I.2: Mobile Application Tasks

Task	Steps
Sign in	Enter the server's address on the site network (for example 192.168.1.20:8000), e-mail and password.
See alerts	Alerts tab: open or all alerts, updated live.
Resolve an alert	Tap an alert to see its snapshot, confidence and model version; Resolve (Safety Officer and Admin).
See the site map	Map tab: zones with head counts, vests and cameras; tap a camera to open it.
Watch a camera	Cameras tab → a camera: live picture with its danger areas (Safety Officer and Admin); Admins can switch an area off.
Check workers	Workers tab: role, zone, vest and presence.
Receive push	Android: allow notifications at first sign-in; tapping a notification opens the alert.
Sign out	Settings tab → Sign out; the phone stops receiving push.

Appendix II

Installation and Setup Guide

Table II.1 brings up the system on one computer, as used for the measurements in Chapter 5. The repository README describes every option and environment variable.

Table II.1: Setup Steps

Step	Action	Commands and notes
1	Prerequisites	Python 3.12 or later, Node.js 18 or later, PostgreSQL 14 or later (18 used); Redis and Mosquitto for multi-process runs; a webcam or IP camera
2	Database	Create user and database sitesafe in PostgreSQL
3	Backend	cd backend; python3 -m venv .venv; pip install -r requirements.txt; cp .env.example .env and set SECRET_KEY and DB_*; python manage.py migrate; python manage.py createsuperuser; daphne -b 127.0.0.1 -p 8000 sitesafe.asgi:application
4	Background jobs	python manage.py run_mqtt_consumer; python manage.py run_presence_monitor; schedule python manage.py purge_alert_snapshots daily
5	Report service	cd report-service; npm ci; DJANGO_API_URL=http://localhost:8000/api/ npm start (port 3001)
6	Web dashboard	cd frontend; npm install; npm run dev; open http://localhost:5173 and sign in
7	Camera token	python manage.py create_service_token CAM-01 --kind CAMERA --camera-id CAM-01 (the key is shown once)
8	Camera pipeline	cd ai-module; pip install -r requirements-vision.txt; copy the weights to models/sitesafe_v5.pt; SITESAFE_DEVICE_KEY=<key> python inference.py --camera-id CAM-01 --source <device or stream URL>. Keys: Q quit, P pause, S test alert
9	Mobile app	cd mobile; npm ci; npx expo start; sign in with the server's LAN address. Push needs a Firebase project and an Android development build
10	Vest simulator	python manage.py create_service_token vest-sim --kind VEST_GATEWAY --simulator; SITESAFE_DEVICE_KEY=<key> python tools/vest_simulator.py tools/vest_scenarios/demo.json --speedup 5
11	Docker (alternative)	cp .env.docker.example .env.docker; docker compose --env-file .env.docker up --build (dashboard on port 8080, API on 8000); not yet run
12	Demo site	python manage.py setup_demo: demo accounts, zones, workers with badges, vests, device keys and a placed camera, no alerts; docs/DEMO_SCRIPT.md walks through a demonstration

Appendix III

API and Database Reference

Tables III.1 and III.2 list the backend's endpoints and database tables described in Section 4.4; Figure 4.2 shows their relations.

Table III.1: Core API Endpoints

Endpoint	Method	Description
/api/token/, /api/token/refresh/	POST	Sign in; renew the access token
/api/v1/profile/, /api/v1/profile/change-password/	GET/PATCH, POST	Signed-in user's profile and password
/api/dashboard/stats/	GET	Counts and charts from stored records
/api/alerts/	GET/POST	List or create alerts (date range, type, status, camera and newest-N filters)
/api/alerts/{id}/, /api/alerts/{id}/resolve/	GET/PATCH, POST	Read, update or resolve one alert
/api/alerts/{id}/snapshot/	GET	Evidence frame of a camera alert; signed-in users only
/api/alerts/simulate/	POST	Create a simulated alert (labelled)
/api/v1/camera-events/	POST	Camera event with event_id; device token
/api/v1/camera-activity/	POST	Badge sightings and idle start/end; device token
/api/v1/camera-config/	GET	Alert rules and danger areas for this camera; device token
/api/v1/camera-frames/ (+ /latest/)	POST, GET	Live-view JPEG from a camera (device token); latest frame for officers
/api/v1/camera-counts/	POST	People in a camera's view, for zone head counts; device token
/api/v1/cameras/, /api/v1/camera-map/ (+ /{id}/)	GET, GET/PUT/DELETE	Cameras with live status; site-map rows and Admin placement
/api/v1/camera-areas/	GET/POST/PATCH/DELETE	Danger areas on camera pictures; Admins write
/api/alerts/ingest/	POST	Legacy edge ingest; device token
/api/workers/, /api/workers/{id}/	GET/POST, GET/PUT/PATCH/DELETE	Worker registry and vest assignment
/api/workers/{id}/badge/	GET/POST	Read or reissue a worker's QR badge
/api/v1/vests/, /api/v1/vests/{id}/	GET/POST, GET/PUT/PATCH/DELETE	Smart Vest records
/api/v1/vests/telemetry/	POST	GPS, battery and SOS telemetry; gateway token
/api/v1/activity/ (+ /timeline/, /daily/, /summary/)	GET	Activity records, per-worker timelines and daily totals
/api/v1/detections/	GET/POST	Detection records
/api/v1/alert-config/	GET/POST/PUT/PATCH/DELETE	Alert rules: enabled, threshold, notify
/api/v1/zones/ (+ /{id}/workers/, /{id}/check_point/)	GET/POST/PUT/PATCH/DELETE	Zones, assigned workers, point-in-zone test
/api/v1/hot-work-permits/ (+ /{id}/revoke/)	GET/POST/PUT/PATCH/DELETE	Hot-work permits and revocation
/api/v1/fcm-tokens/, /api/v1/fcm-tokens/{token}/	POST, DELETE	Register or remove a phone's push token
/api/v1/reports/ (+ /generate/, /{id}/download/)	GET, POST, GET	List, generate (DOCX or PDF) and download reports

Endpoint	Method	Description
/ws/dashboard/?token=<JWT>	WebSocket	Live alert, vest, worker, zone and camera events
/ws/cameras/?token=<JWT>	WebSocket	Live camera frames for Safety Officers and Admins

Table III.2: Database Schema Summary

Table	Key fields	Relations / notes
users	id, email, full_name, role, is_active	Roles: Admin, Safety Officer, Viewer
service_tokens	id, name, kind, key_hash, camera_ids, is_simulator, is_active	Device tokens; only a hash of the key is stored
workers	id, name, role, zone_id, badge_code, last_seen_at, location_zone_id, location_at, is_active	FK zones; badge_code is unique (QR payload); last located zone
smart_vests	id, vest_id, worker_id, battery_level, latitude, longitude, sos_active, last_seen	One-to-one with workers; battery unknown until reported
zones	id, name, risk_level, boundaries, camera_ids, max_occupancy, camera_views_overlap	Polygon boundaries and camera assignment (JSON); capacity state
cameras	id, camera_id, people_count, counted_at, latitude, longitude, heading, field_of_view, view_range_m	Latest people count and site-map placement
camera_areas	id, camera_id, name, polygon, severity, is_active	Danger areas; corners as fractions of the picture
hot_work_permits	id, zone_id, description, starts_at, ends_at, issued_by_id, revoked_at	FK zones, users
alerts	id, alert_type, severity, source, worker_id, camera_id, hot_work_permit_id, confidence, model_version, snapshot, event_id, is_resolved	FK workers, permits; event_id unique for idempotent ingest
alert_config	id, zone_id, alert_type, is_enabled, threshold_seconds, notify_push, notify_email	One rule per type and zone; SOS and FIRE always enabled
activity_logs	id, worker_id, zone_id, action, timestamp, metadata	Check-in/out, idle and zone events at event time
detections	id, camera_id, worker_id, class_name, confidence, bbox	FK workers
reports	id, report_type, file_format, date_from, date_to, includes_evidence, created_by_id	FK users; stored before download
fcm_tokens, user_preferences	user_id, token, device_name / notify_push, notify_email	FK users; push targets and opt-outs

Appendix IV

Test Evidence

Tables IV.1 to IV.3 give the evidence summarised in Chapter 5: the status reached by each functional requirement, the end-to-end verification record and the defects found by testing.

Table IV.1: Requirements, Evidence and Status

ID	Requirement	Evidence	Status
FR-01	Sign-in and silent session renewal	Account tests; browser: corrupted token renewed, requests retried	Integrated
FR-02	Roles enforced by the API	16 permission tests	Tested
FR-03	UI shows only permitted actions	Browser check as Safety Officer and Admin; Viewer not checked in the browser	Integrated
FR-04	Detect missing PPE (10 classes)	Held-out mAP@0.5 0.562 on a leaky split; no sealed target-camera set	Coded
FR-05	One alert per person per violation episode	31 tests; clip with 2 people without helmets gives 2 alerts; live webcam runs fixed duplicate-box and moving-head repeats	Tested
FR-06	Falls from camera pose	24 tests; no fall video evaluated	Tested
FR-07	Worker-vehicle proximity	18 tests; looped test clip 10 alerts to 2, one per person	Tested
FR-08	Inactivity with dashboard-set timeout	10 tests; rule polling tests	Tested
FR-09	Fire/smoke with hot-work permits	20 tests; model trained, held-out mAP@0.5 0.787; permit downgrade end to end	Integrated (images only)
FR-10	QR worker identification	21 tests; synthetic frames and a test video with a printed badge	Integrated (test video)
FR-11	Reliable camera event delivery	8 outbox tests; 100-event outage (Table 5.3)	Integrated
FR-12	Only authorised devices submit data	Ingest tests: anonymous, revoked, wrong-kind, wrong-camera tokens	Tested
FR-13	Live updates for every change	Realtime tests; two clients converge; browser	Integrated
FR-14	Authenticated WebSocket (4401)	Realtime tests for missing, bad and expired tokens and origin	Tested
FR-15	Alert rules change runtime behaviour	Ingest tests: disabled type, notify flags, SOS always on	Tested
FR-16	Push notifications	Mocked FCM and app registration tests; no phone yet	Coded
FR-17	E-mail notifications	Tests with a local mail backend	Tested
FR-18	Worker registry in the UI	7 tests; browser registration with vest	Integrated
FR-19	Zones with polygons and cameras	Zone tests; polygon drawn and saved in the browser	Integrated
FR-20	Automatic breach and over-occupancy alerts	9 tests; telemetry into a restricted zone end to end	Integrated (simulated GPS)
FR-21	Smart Vest GPS, SOS and link	Simulator scenario end to end; hardware not built	Software integrated; hardware deferred
FR-22, 23	Telemetry validation; vest offline	Validation and presence tests; browser occupancy timeout	Tested; Integrated
FR-24	Activity history: check-in/out, idle	23 tests; timeline and PDF agree end to end	Integrated
FR-25	Incident list, resolve, export	Browser: every filter, resolve broadcast, CSV export, responsive table	Integrated
FR-26	Dashboard shows recorded data only	3 tests; browser reconciliation	Integrated
FR-27	DOCX and PDF reports	19 tests; both types and formats end to end	Integrated
FR-28	Mobile app	25 tests; web build: alerts, map, cameras, live picture end to end	Integrated on web; no device

ID	Requirement	Evidence	Status
FR-29	Alert evidence (snapshot, model version)	30 tests; live runs with test videos: frame and model on web, mobile and reports	Integrated (test videos)
FR-30	Live camera grid	18 tests; test video and synthetic frames live at 5 frames/s; Viewer sees none	Integrated (test video)
FR-31	Camera-based location and head counts	31 tests; badge clip places the worker; crowd clip head counts with hysteresis	Integrated (test videos)
FR-32	Danger areas on the camera picture	22 tests; area drawn live, alert with snapshot and badge within 15 s	Integrated (test video)
FR-33	Cameras on the site map	11 tests; placement; live, offline and alerting states; role checks	Integrated (test video)
FR-34	Evidence snapshots in reports	13 tests; incident PDF with 24 figures, safety DOCX by zone	Integrated (test video)

Table IV.2: End-to-End Verification Record

Date	Scenario	Observed result	Boundary
16 Oct	Token renewal in the browser	Corrupted access token renewed; requests retried with 200	Single host
16 Oct	Two dashboard clients	Both received create, create and resolve in order	Single host
16 Oct	Outbox replay after outage	3 events queued: 2 stored, 1 dead-lettered (wrong camera)	Small fixture; see Table 5.3
16 Oct	Restricted-zone breach	Telemetry into a restricted zone raised a live ZONE_BREACH	GPS simulated over HTTP
16 Oct	QR attribution	Badge decoded; alert attributed to its holder; second person not identified	Synthetic frame; no physical camera
16 Oct	Reports	Both report types in DOCX and PDF through the API; PDF inspected	Test data
16 Oct	Mobile app (web build)	Sign-in, live alert, detail, resolve, cross-session resolve, workers, sign-out	No phone; no push
16 Oct	Vest simulator scenario	Simulated breach and SOS shown; vest offline after silence	Simulator, labelled
20 Oct	Hot-work permit	Fire in permitted zone MEDIUM and linked; elsewhere CRITICAL	Injected events; no fire model
20 Oct	Attendance	60 min on site, 20 min idle, check-out at last sighting; timeline equals PDF	Synthetic camera events
20 Oct	Alert evidence	Snapshot sent through the outbox; one image stored, a replay stored none; frame and model version shown on web and mobile; retention removed the image and kept the alert	Synthetic frame; no live camera run
Earlier	Proximity, real detector	Still image: 4 people, 1 vehicle, CRITICAL alert stored and shown	Still image only
27 Oct	Live camera grid	Test video and synthetic frames live at 5 frames/s; offline camera shown; Viewer saw no video	Test video; no site camera
27 Oct	Camera location and head counts	Badge clip placed the worker in its camera's zone; one over-capacity alert per crowd period	Test videos
27 Oct	Danger area	Area drawn on the live view; alert with snapshot and badge holder; red fill and banner; Safety Officer read-only	Test video
27 Oct	Cameras on the site map	Placement; live, offline and alerting states; live thumbnail for officers, none for Viewers	Test video
27 Oct	Report evidence	Incident PDF with 24 captioned figures referenced from the log; safety DOCX by zone	Test-video frames

Date	Scenario	Observed result	Boundary
27 Oct	Mobile map and cameras	Map and live picture with area outline; an area switched off on the phone stopped at the camera within 15 s	Web build at phone size

Table IV.3: Defects Found by Testing

Defect	Found by	Fix
Every vest telemetry message failed on PostgreSQL (row lock across a nullable join)	Running the backend tests on PostgreSQL	Lock only the vest row
Outage recovery took 60.3 s, over the 60 s target	Outage measurement (Table 5.3)	Retry ceiling lowered from 60 s to 30 s
Full pipeline ran at 9.8 FPS because Ultralytics defaulted to the CPU	Throughput measurement (Table 5.4)	Automatic device selection uses the GPU
A late camera event created a second check-in	Activity tests	Late events merged into their session
A migration left 100% battery on vests that never reported	Migration check on PostgreSQL	Battery shown as unknown until reported
Browsers saw close code 1006, not 4401, for a bad token	WebSocket tests in the browser	Accept, then close with 4401
Report tests were silently not run	Review of test counts	Missing package file added
Camera-to-zone lookup used a PostgreSQL-only JSON operator	Tests on SQLite	Portable lookup
Fire images whose labels were all invalid became false negatives	Dataset preparation tests	Such images skipped and reported
One person got repeat PPE alerts: duplicate same-class boxes became separate episodes	Live webcam run (55 s)	Overlapping or nested same-class boxes merged; episode close time 10 s to 30 s
A timeline test failed before noon: its shift was partly in the future	Re-running the suite in the morning	Test uses yesterday's shift
The mobile app could not load alert snapshots: the API refused Accept: image/* with 406	End-to-end check of the mobile web build	The snapshot endpoint ignores the Accept header
Every worker update closed dashboard sockets: a raw date-time in the payload	Live camera-location run	Times sent as ISO text; payload test
A noisy head count raised 5 over-capacity alerts in 4 minutes	Live crowd-video run	Hysteresis: 10 s to alert, 60 s within the limit to re-arm
One worker beside a flickering car raised 14 proximity alerts in 3 minutes	Live danger-area run	One encounter per person; camera-wide cooldown removed
A different person at the same spot 14 s later was taken for a re-found one	Replay of the 40-scene test video	Re-finding limited to 5 s
Every report downloaded under a generic file name	Live report run	The API exposes the file-name header to the dashboard

References

[1] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.

[2] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 7464–7475.

[3] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics YOLO," version 8.4.41, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] testcasque, "PPE Detection (ppe-detection-qlq3d)," Roboflow Universe, dataset version 1, CC BY 4.0, exported 19 Oct. 2024. [Online]. Available: https://universe.roboflow.com/testcasque/ppe-detection-qlq3d

[5] H. Nath, F. Alsharef, and M. Nahangi, "Deep learning-based personal protective equipment detection on construction sites: A review," Autom. Constr., vol. 146, p. 104671, 2023.

[6] S. Fang, H. Li, and S. K. Ong, "Computer vision for construction safety: A systematic review," Saf. Sci., vol. 158, p. 105990, 2023.

[7] M. Awolusi, E. Marks, and M. Hallowell, "Wearable technology for personalized construction safety monitoring: A review of sensor-based and vision-based approaches," Autom. Constr., vol. 125, p. 103622, 2021.

[8] W. Fang et al., "Falls from heights: A computer vision-based approach for safety harness detection," Autom. Constr., vol. 109, p. 103013, 2020.

[9] T. Li, D. Luo, and J. Gong, "Real-time PPE compliance detection using edge computing and deep learning on construction sites," J. Comput. Civ. Eng., vol. 37, no. 3, 2023.

[10] Django Software Foundation, "Django REST Framework," version 3.17.1, 2026. [Online]. Available: https://www.django-rest-framework.org/

[11] Meta Platforms, "React Native," version 0.88, 2026. [Online]. Available: https://reactnative.dev/

[12] Google, "Firebase Cloud Messaging," 2024. [Online]. Available: https://firebase.google.com/docs/cloud-messaging

[13] Espressif Systems, "ESP32 Technical Reference Manual," version 5.0, 2023. [Online]. Available: https://www.espressif.com/en/products/socs/esp32

[14] Semtech, "LoRa and LoRaWAN: A Technical Overview," 2023. [Online]. Available: https://lora-developers.semtech.com/

[15] Eclipse Foundation, "Mosquitto MQTT Broker," version 2.0, 2023. [Online]. Available: https://mosquitto.org/

[16] International Labour Organization, "Safety and health at work," 2023. [Online]. Available: https://www.ilo.org/global/topics/safety-and-health-at-work

[17] Pakistan Bureau of Statistics, "Labour Force Survey 2022-23," Islamabad, Pakistan, 2023.

[18] SafetyCulture, "iAuditor Mobile Safety Platform," 2024. [Online]. Available: https://safetyculture.com/

[19] A. Siddiqui, H. Akhtar, and S. Ahmed, "IoT-based construction site monitoring system: A framework for developing countries," in Proc. IEEE Int. Conf. Emerg. Technol. (ICET), Islamabad, Pakistan, 2022, pp. 1–6.

[20] Y. Lecun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[21] Expo, "Expo SDK 58," 2026. [Online]. Available: https://docs.expo.dev/

[22] Django Software Foundation, "Channels," version 4.3.2, 2026. [Online]. Available: https://channels.readthedocs.io/

[23] Leaflet, "Leaflet: an open-source JavaScript library for interactive maps," version 1.9.4, 2023. [Online]. Available: https://leafletjs.com/